

ON-CAMPUS WEARABLE AI-ENABLED SECURITY BRACELET SYSTEM



LEBOPO HAILEY MAKANE

202108391

GORATA MATSAPA

202101679

**DEPARTMENT OF ELECTRICAL ENGINEERING
FACULTY OF ENGINEERING AND TECHNOLOGY
UNIVERSITY OF BOTSWANA**

Submitted in partial fulfilment of the requirements for degree of
Bachelor of Engineering

MAY 2026

Declaration

We certify that this is a bona fide project work and report EEB540/550 of:

Student Name: _____

ID No: _____

Student Name: _____

ID No: _____

Project Title: _____

Required for the successful completion of the Bachelor of Engineering Degree
(Electronics/Electrical)

Date of Completion of the Report: _____

Student Signature: _____

Student Signature: _____

Acknowledgements

We would like to express our sincere gratitude to God for the strength and perseverance granted to us throughout the development of this project. We extend our heartfelt appreciation to our project supervisor Dr Kumbirayi Nyachionjeka for the continuous guidance and support provided throughout this research and development process. We also acknowledge the Faculty of Engineering and Technology, particularly the Department of Electrical and Electronics Engineering, for providing workspace and technical resources necessary to carry out this project. Special thanks are extended to the Department of Tertiary Education Financing for the financial support that contributed towards the realization of this project. We are equally grateful to ED-TECH and UniPod for their assistance with resources and 3D printing services which supported the physical development of our prototype. Finally, we appreciate everyone who contributed directly or indirectly to the successful completion of this project.

TABLE OF CONTENTS

LIST OF TABLES	viii
LIST OF FIGURES	ix
ABBREVIATIONS AND ACRONYMS.....	xi
ABSTRACT.....	xii
CHAPTER ONE	1
INTRODUCTION.....	1
1.1 Background of the Study.....	1
1.2 Problem Statement	2
1.3 Motivation	2
1.4 Significance of Project	2
1.5 Objectives.....	3
1.5.1 Main Objective	3
1.5.2 Specific Objectives	3
1.6 Scope of the Study.....	4
1.7 Report organization	4
CHAPTER TWO	5
LITERATURE REVIEW	5
2.1 Introduction	5

2.2 Overview of Campus Safety Challenges	6
2.3 Existing Safety Solutions	6
2.4 Technologies Behind Safety Systems	7
2.5 AI and ML in Wearable Safety Devices	8
2.6 Challenges and Limitations	8
2.7 Gaps and Need for Improvement	9
CHAPTER THREE	10
METHODOLOGY	10
3.1 Introduction	10
3.2 System Architecture	10
3.3 Hardware System Implementation	12
3.3.1 Bracelet Hardware Design	12
3.3.2 Clip-on Camera Integration	16
3.4 Software System Implementation	18
3.4.1 Cloud Database and Communication Layer	18
3.4.2 Security Dashboard Development	20
3.4.3 Student Mobile Application Development	22
3.4.4 Telegram Patrol Bot Integration	23
3.4.5 AI Threat Prioritization Model	25

3.4.6 Hybrid Location Mapping System	29
3.5 Testing And Validation	30
3.5.1 Alert Trigger Testing	30
3.5.2 Database Synchronization Testing	31
3.5.3 Communication Testing	31
3.5.4 AI Prediction Testing.....	32
3.6 Ethical Considerations.....	32
3.7 Limitations	33
CHAPTER FOUR.....	34
RESULTS AND DISCUSSION	34
4.1 Introduction	34
4.2 Hardware Prototype Results.....	34
4.2.1 Wearable Bracelet results	34
4.2.2 Clip-on Camera Results.....	35
4.3 End-to-End System Results.....	37
4.3.1 Firebase logging	37
4.3.2 Security Dashboard Response	38
4.3.3 Telegram Patrol Bot Response	40
4.3.4 Student App Response.....	41

4.3.5 Incident History and Data Management.....	44
4.3.6 AI Priority Scoring Results.....	46
4.4 Discussion	47
4.4.1 System Integration and Functional Performance.....	47
4.4.2 Real-Time Synchronization and Latency Performance.....	47
4.4.3 Hybrid Location Mapping Performance.....	48
4.4.4 Platform Performance and User Interface Functionality	49
4.4.5 Prototype Limitations and Future Improvements	50
CHAPTER FIVE	51
CONCLUSION AND RECOMMENDATIONS.....	51
5.1 Conclusion.....	51
5.2 Recommendations for Future Study.....	52
REFERENCES.....	53
APPENDICES	55
APPENDIX A-1: GANTT CHART.....	55
APPENDIX A-2: BUDGET.....	56
APPENDIX B: PYTHON CODES.....	59
APPENDIX B-1: Dashboard Active Alert Retrieval and Priority Classification	59
APPENDIX B-2: Student Mobile Application Real-Time Data Synchronization.....	61

APPENDIX B-3: Telegram Bot Active Alert Retrieval.....	62
APPENDIX B-4: Hybrid Location Mapping and Geofencing Logic.....	64
APPENDIX B-5: AI Threat Detection and Threat Scoring Code Snippets	65
APPENDIX C: ARDUINO IDE (.ino) CODES	66
APPENDIX C-1: ESP32-CAM Image Capture and Cloud Upload Code Snippet	66
APPENDIX C.2: ESP32 DevKit Emergency Trigger Detection and Alert Workflow	69

LIST OF TABLES

Table 1 List of components required and budget	56
---	-----------

LIST OF FIGURES

Figure2. 1 Evolution of Campus and Personal Safety Measures.....	5
Figure2. 2 Comparison of Manual, Voice-Activated, and AI-Enabled Wearable Safety Systems	7
Figure2. 3 ESP32-Based Portable GPS Tracker with GSM Communication and Motion Sensing [13].....	8
Figure2. 4 Conceptual Architecture of the Proposed Wearable Security System	9
Figure3. 1 Block Diagram of Overall System Architecture	10
Figure3. 2 Flowchart of the overall system	11
Figure3. 3 SOS Bracelet- Hardware and system logic block diagram	13
Figure3. 4 Assembled bracelet prototype on a DIY prototype board	15
Figure3. 5 Clip-on Camera Operational Logic Flow	17
Figure3. 6 Firebase Realtime Database JSON Tree Structure	20
Figure3. 7 Security Dashboard Interface	21
Figure3. 8 Student Mobile Application Home Interface	23
Figure3. 9 Telegram Patrol Bot	24
Figure3. 10 The AI processing workflow	26
Figure 4. 1 Final Prototype of Bracelet.....	35

Figure 4. 2 Clip-on Camera prototype	36
Figure 4. 3 Picture captured by Clip-on camera showing in the database storage bucket.....	37
Figure 4. 4 Alert logged at the cloud database.....	38
Figure 4. 5 Alert being shown in the security dashboard's campus map	39
Figure 4. 6 Alert priority and more details being shown in the active alerts list	39
Figure 4. 7 Alert being acknowledged in telegram bot.....	40
Figure 4. 8 Google Maps link opened from the telegram bot.....	41
Figure 4. 9 Warnings showing in student app.....	42
Figure 4. 10 Campus Pulse in student app showing current safety status of campus	43
Figure 4. 11 Anonymous tipoff being sent from student app	43
Figure 4. 12 Same anonymous tipoff from student app being displayed in security dashboard ..	44
Figure 4. 13 Resolved alert is moved to history section	45
Figure 4. 14 Weekly trends of all alerts per zone	45
Figure 4. 15 AI priority scoring results.....	46
 Figure A - 1 Gantt chart showing project key stages and timeline.....	 55

ABBREVIATIONS AND ACRONYMS

AI	Artificial Intelligence
CAD	Computer-Aided Design
CNN	Convolutional Neural Network
CCTV	Closed-Circuit Television
DIY	Do It Yourself
ESP	Espressif Systems microcontroller (commonly ESP32/ESP8266)
5G	Fifth Generation
GPS	Global Positioning System
I2C	Inter-integrated Communication
IoT	Internet of Things
ML	Machine Learning
MPU	Motion Processing Unit
PCB	Printed Circuit Board
SSID	Service Set Identifier (Wi-Fi network name)
Wi-Fi	Wireless Fidelity
YOLO	You Only Look Once

ABSTRACT

Campus safety remains a growing concern in higher education institutions. Poorly monitored areas especially, are areas where students may be vulnerable to theft, harassment, or physical assault. This project presents the design and development of SafeCampus, an on-campus wearable AI-enabled security bracelet system aimed at improving real-time emergency response at the University of Botswana. The system integrates a wearable SOS bracelet with manual and automatic alert triggering, a clip-on camera module, hybrid GPS and Wi-Fi location mapping, cloud-based data synchronization, a Streamlit security dashboard, a Kivy student mobile application, Telegram bot integration and an AI model for image analysis and priority scoring. When an alert is triggered, incident data and location information are transmitted to a Firebase cloud platform for storage and processing. The data is also distributed across connected interfaces. System testing demonstrated successful real-time alert synchronization, location tracking, alert threat prioritization and multi-platform visualization with alerts being sent within acceptable response times under stable network conditions. The results indicate that integrating wearable technology, cloud computing, and intelligent alert prioritization can improve campus emergency monitoring and response efficiency. This provides a scalable framework for enhancing student safety.

[1]

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Campus safety remains a pressing concern in higher education institutions worldwide due to increasing incidents of theft, harassment, assault and other security threats affecting students on and around campuses. These risks are higher in isolated pathways, poorly lit areas and zones with limited security presence. Such incidents impact students' physical safety, academic performance, mental well-being and overall campus experience [1].

Traditional security measures including patrols, CCTV, emergency hotlines, and manual reporting are mostly reactive. They rely heavily on victims manually alerting authorities and communicating their location which is often difficult during distress [1]. In large campus environments delays in response and challenges in locating individuals reduce effectiveness.

Advances in wearable technology, IoT, cloud computing and AI have enabled smarter personal safety solutions [2], [3]. Wearable devices with sensors can monitor activity continuously and connect to cloud platforms in real time. AI enhances detection by analyzing motion patterns to automatically identify distress situations such as sudden impacts, abnormal movement or prolonged stillness going beyond manual activation alone [3].

Cloud integration further supports centralized communication, allowing instant sharing of alerts, location data and evidence across dashboards and mobile apps [4]. This project, SafeCampus: On-Campus Wearable AI-Enabled Security Bracelet System, applies these technologies to the University of Botswana context. It integrates an ESP32-based bracelet with MPU6050 motion sensor for automatic and manual alerts, hybrid GPS/Wi-Fi tracking, and an ESP32-CAM module for visual evidence.

The system uses Firebase as the central platform linking the wearable, AI processing (CNN and YOLO models for threat classification), Streamlit security dashboard, Telegram bot and Kivy

student mobile app. By combining wearable sensors, AI-assisted analysis, cloud connectivity and hybrid positioning, SafeCampus aims to deliver faster emergency response, better situational awareness, and a scalable safety framework for university environments [5].

1.2 Problem Statement

Students at the University of Botswana face increasing safety risks including theft, harassment and physical assault especially in isolated or poorly monitored campus areas. Existing measures such as security patrols and emergency hotlines are largely reactive often leading to delayed responses and inaccurate location tracking. Current wearable devices rely heavily on manual activation which may not be feasible during distress situations. These gaps highlight the need for an intelligent, real-time campus safety system with automated emergency detection, rapid communication and reliable location tracking.

1.3 Motivation

The need for faster and more reliable campus emergency response systems drove this project. Traditional approaches offer limited real-time monitoring and depend mainly on manual reporting. Advances in wearable technology, IoT, cloud computing and AI now allow for a smarter, proactive solution. This project aims to enhance student safety by integrating automated distress detection, hybrid location tracking, and real-time communication into a unified campus security system.

1.4 Significance of Project

This project contributes to the improvement of campus safety by providing a real-time AI-enabled wearable emergency response system tailored for university environments. The integration of IoT, cloud computing, hybrid location tracking, and AI-assisted threat prioritization enhances emergency detection and response efficiency. The system also improves communication between students and campus security personnel while supporting faster incident awareness and decision-making. Additionally, the project demonstrates the practical application of embedded systems and intelligent monitoring technologies in smart campus security solutions.

1.5 Objectives

1.5.1 Main Objective

To develop an On-Campus Wearable AI-Enabled Security Bracelet system that detects distress using motion analysis and transmits emergency alerts, location information, and trigger-based evidence to a cloud-based security platform for real-time campus emergency response at the University of Botswana.

1.5.2 Specific Objectives

1. To design and integrate a wearable safety bracelet with motion sensors, a clip-on body camera, and a manual emergency trigger for distress detection and evidence capture.
2. To develop a cloud-based communication and storage platform for processing emergency alerts, storing incident data, and supporting real-time synchronization between connected system modules.
3. To implement a hybrid location tracking system using GPS and Wi-Fi-based zone mapping to improve user localization across indoor and outdoor campus environments.
4. To develop a security dashboard, Telegram patrol notification system, and student mobile application for real-time alert monitoring, emergency communication, and campus safety reporting.
5. To optimize the system for low-power operation, reliability, and real-time performance through testing and validation under simulated campus emergency scenarios.

1.6 Scope of the Study

This study focuses on the design, prototyping and testing of the On-Campus Wearable AI-Enabled Security Bracelet system within the University of Botswana campus environment. The scope includes the integration of hardware components such as the ESP32-based wearable device, motion sensors, GPS module, Wi-Fi communication system and clip-on body camera for emergency monitoring and trigger-based evidence capture.

The study also covers the development of software components including cloud-based data communication using Firebase Realtime Database and Storage, AI-assisted alert prioritization, hybrid GPS and Wi-Fi-based location mapping, a Streamlit-based security dashboard, a Telegram patrol notification bot and a Kivy-based student mobile application. The system's functionality is evaluated through simulated campus emergency scenarios to assess real-time alert transmission, synchronization, location tracking and overall responsiveness.

The project is limited to prototype-level implementation and testing and does not include full-scale campus deployment, integration with institutional security infrastructure or long-term user adoption studies beyond the project evaluation period.

1.7 Report organization

Chapter 1 discusses the introduction to the project and the project background. Chapter 2 discusses the literature review of existing and related technologies as well as studies relevant to the system. Chapter 3 describes the methodology used in the design and implementation of the prototype which includes both hardware and software. Chapter 4 presents the results obtained from system testing and discusses the performance of the developed prototype. Chapter 5 provides the conclusions drawn from the study and outlines recommendations for future improvements. Next the references list all sources used in the development of this project. Lastly the appendices include supporting materials such as source code and additional documentation.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing literature on wearable safety technologies, IoT-based emergency systems, AI-enabled distress detection, cloud communication platforms and campus safety monitoring systems. The review covers technologies and approaches relevant to the proposed On-Campus Wearable AI-Enabled Security Bracelet system.

It examines current campus safety challenges, available wearable solutions, supporting hardware and communication technologies, AI applications in wearables as well as limitations of existing systems. Emphasis is placed on motion-based distress detection, hybrid location tracking, cloud-integrated emergency systems and real-time monitoring platforms.

The purpose is to identify gaps in current solutions that support the need for this project and to provide the technical foundation for design decisions such as ESP32 hardware, Firebase cloud integration, hybrid GPS/Wi-Fi tracking, AI alert prioritization and the dashboard/mobile app interfaces.

Figure 2.1 illustrates the evolution of campus and personal safety technologies from traditional reactive systems to modern AI-enabled wearable solutions.



Figure2. 1 Evolution of Campus and Personal Safety Measures

2.2 Overview of Campus Safety Challenges

University campuses continue to face safety issues such as theft, harassment, assault and medical emergencies especially during late hours and in isolated areas [3]. These incidents affect student wellbeing, academic participation and overall sense of security with female students and those moving alone at night often more vulnerable [14].

Traditional measures like patrols, CCTV and emergency contact systems are mainly reactive [1]. Delays in communication, limited situational awareness and inaccurate location tracking reduce response effectiveness. Indoor environments further complicate positioning due to weak GPS signals [4].

Modern campus safety systems increasingly require proactive, automated solutions for rapid detection and real-time information sharing [5]. The integration of wearables, IoT and cloud platforms has become key to improving emergency response in educational settings [8].

2.3 Existing Safety Solutions

Wearable safety systems have progressed from basic panic-button devices to more advanced connected platforms [5]. Early devices depended on manual activation which failed when users were incapacitated or unable to respond [6, 14].

Mobile safety apps later added GPS sharing and live tracking [7] but they suffer from battery drain, network issues and delayed use in stressful situations.

Recent solutions incorporate IoT, cloud connectivity and sensor monitoring [8]. AI-enabled wearables now use accelerometers and motion analysis to automatically detect distress patterns such as sudden impacts or prolonged stillness, reducing reliance on manual triggers [5]. Figure 2.2 summarizes the progression from manual to AI-enabled wearable safety systems.

Key Features / AI-Enabled		
 Manual	 Voice-Activated	 AI-Enabled
SOS button on a wearable	Enables voice commands	Detects distress events
Triggered by user pressing button	Triggered by voice command	Triggered by movement analysis
May be linked to a mobile app	Integrated with voice recognition	Incorporates sensors and AI
Relies on conscious user action	Usability challenges for some users	Limited availability on the market

Figure2. 2 Comparison of Manual, Voice-Activated, and AI-Enabled Wearable Safety Systems

2.4 Technologies Behind Safety Systems

Modern wearable safety systems integrate sensors, wireless protocols, cloud platforms and microcontrollers [8]. GPS is commonly used for outdoor tracking but performs poorly indoors, leading to hybrid approaches combining Wi-Fi localization and Assisted GPS [4].

Communication technologies such as Wi-Fi, Bluetooth Low Energy (BLE) and GSM enable data transmission. Wi-Fi is particularly suitable for campuses due to existing infrastructure [1]. Accelerometers and gyroscopes detect abnormal movement while ESP32 microcontrollers provide efficient sensor integration, wireless connectivity and low power use [13]. Figure 2.3 shows an ESP32-based portable GPS tracker with motion sensing and wireless communication similar to this project.

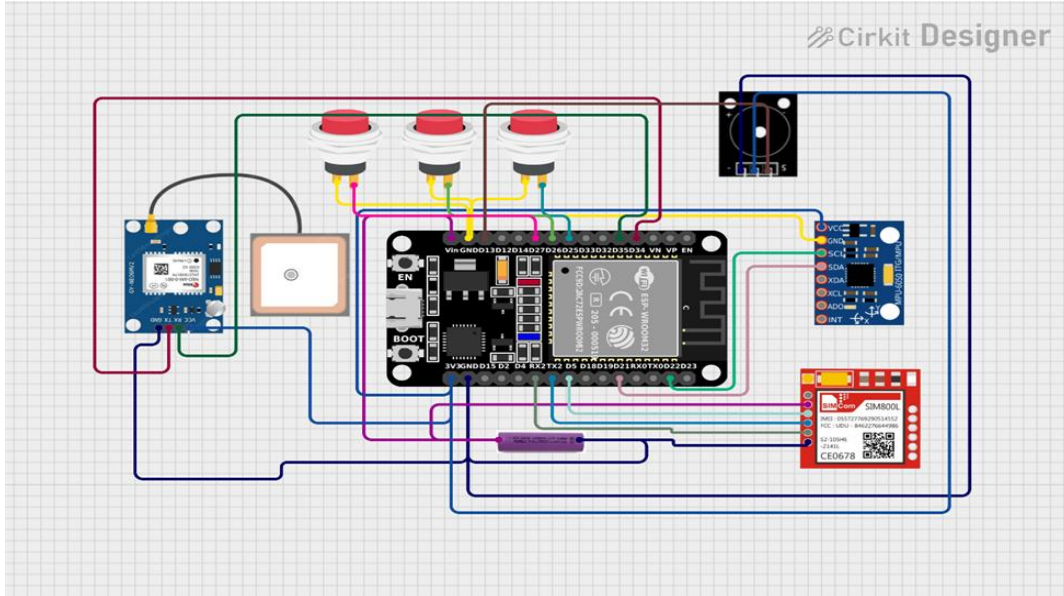


Figure2. 3 ESP32-Based Portable GPS Tracker with GSM Communication and Motion Sensing [13]

2.5 AI and ML in Wearable Safety Devices

AI and Machine Learning have enhanced wearable safety by enabling automatic detection of distress through motion data analysis [2, 5]. Algorithms such as SVM, decision trees and neural networks classify abnormal patterns with good accuracy in controlled settings [9].

Due to limited device processing power, many systems send data to the cloud for AI analysis, improving scalability and real-time synchronization [8, 12]. AI also supports alert prioritization based on severity [5].

2.6 Challenges and Limitations

Key challenges include unreliable connectivity indoors, high power consumption from sensors and cameras as well as privacy concerns over location and media data [4, 8, 11]. AI models may produce false positives in real-world conditions due to limited datasets and environmental variations [9]. Many systems also lack full integration across devices, cloud, dashboards and apps [6].

2.7 Gaps and Need for Improvement

Existing systems often rely heavily on manual activation, lack reliable indoor positioning and provide limited integration between wearables, cloud platforms, AI prioritization, evidence capture and user interfaces [4, 6, 8].

The proposed SafeCampus system addresses these by combining automated motion detection, hybrid GPS/Wi-Fi tracking, Firebase synchronization, AI-assisted prioritization (CNN and YOLO), camera evidence capture, Streamlit dashboard, Telegram notifications and Kivy student app.

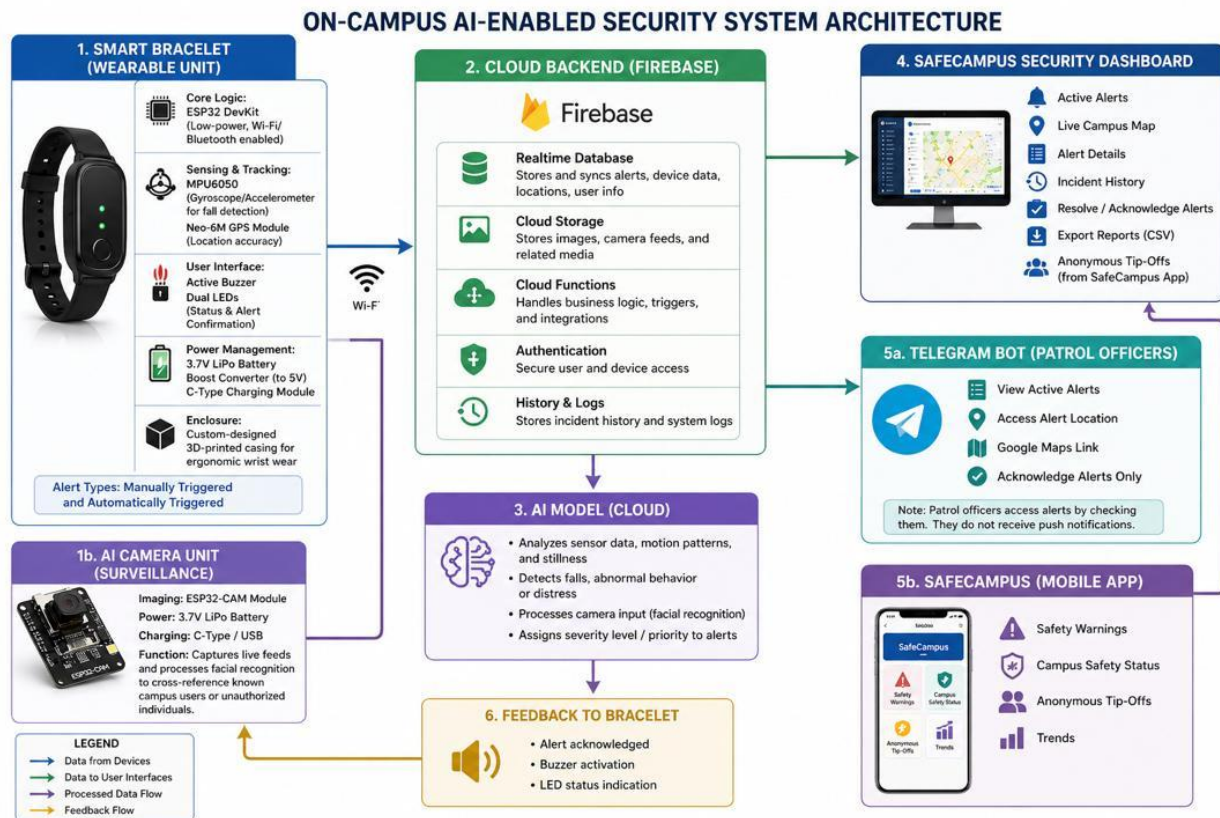


Figure2. 4 Conceptual Architecture of the Proposed Wearable Security System

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter presents the methodology used in the design, implementation, and testing of the AI-enabled campus safety wearable system. The project used a prototype-based system development approach. This involved iterative design, simulation, software implementation and functional testing to achieve the expected objectives. The system architecture employed had a wearable safety bracelet with a clip-on camera, cloud database, AI threat prioritization model, campus security dashboard, Telegram patrol bot and a student mobile application. The chapter further explains the overall system design, development approach, implementation process, testing procedures, ethical considerations, and limitations encountered during the project

3.2 System Architecture



Figure3. 1 Block Diagram of Overall System Architecture

Device Layer: This layer comprises the hardware components. These include a smart wearable bracelet and a clip-on camera. This layer is responsible for alert triggering (either manual or autonomous) and the simultaneous capture of the visual evidence by the clip-on camera. Both bracelet and clip-on camera then transmit telemetry data, and visual evidence to the cloud database.

Processing Layer: This layer acts as the central hub of the system. It is made up of the cloud database which stores and processes data and the AI priority scorer, which uses the visual evidence captured to categorize manual alerts based on severity. This layer is also responsible for the logic for high-priority emergency routing and coordinates the feedback loop.

Application Layer: This layer facilitates user interaction through a Security Dashboard, Telegram Patrol Bot, and Student App. These interfaces use bidirectional communication to retrieve real-time alert data and transmit relevant data such as acknowledgment back through the processing layer to the end-user.

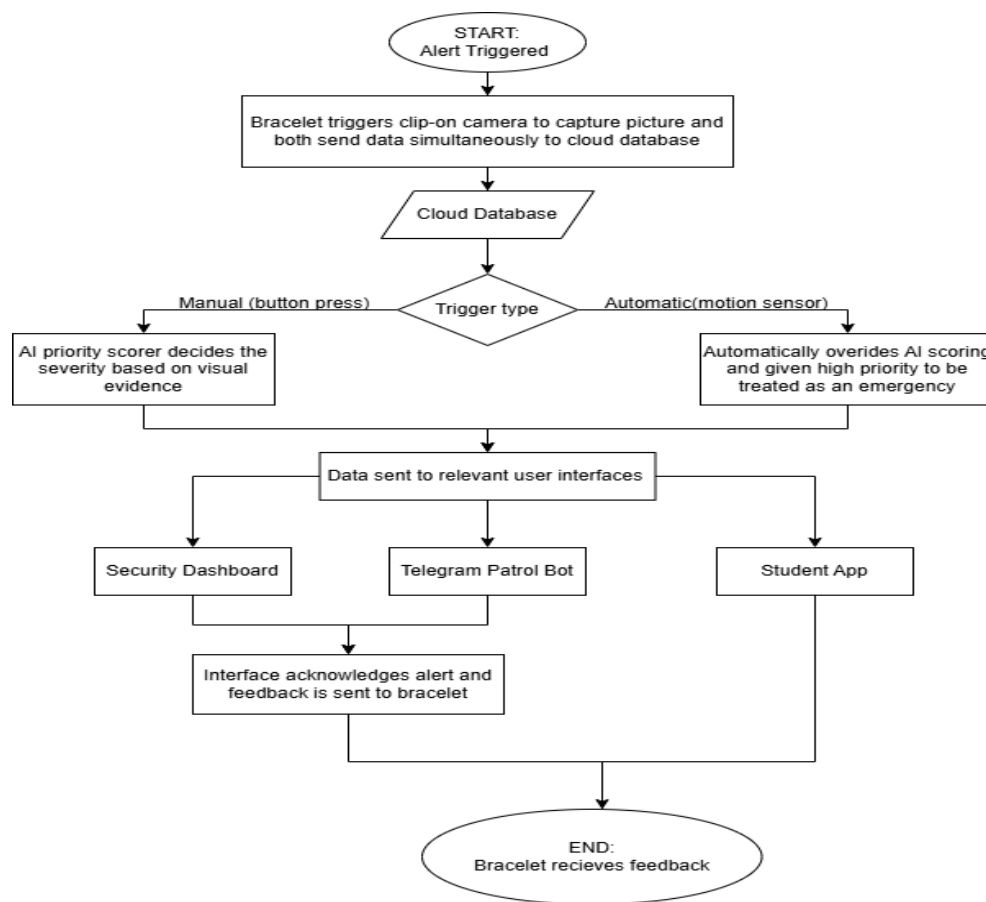


Figure3. 2 Flowchart of the overall system

3.3 Hardware System Implementation

3.3.1 Bracelet Hardware Design

The wearable bracelet serves as the core hardware component of the SafeCampus system, handling emergency detection, sensor data collection, wireless communication and alert transmission. The design prioritizes compactness, portability, low power consumption and reliable real-time performance for daily campus use.

The hardware is built around an ESP32 microcontroller platform, chosen for its built-in Wi-Fi, low-power operation, adequate processing power and strong IoT compatibility. The ESP32 acts as the central processor managing all connected modules and cloud interactions.

Key integrated components include:

1. MPU6050 motion sensor for detecting movement and prolonged stillness
2. GPS module for outdoor location tracking
3. Wi-Fi module for cloud connectivity and indoor localization fallback
4. Emergency SOS button for manual alerts
5. Active buzzer for feedback notifications and local alarms
6. Power management circuitry with battery support
7. Clip-on body camera interface for evidence capture during emergencies

Figure 3.3 shows brief summary of the hardware implementation of the system in the form of a logic flow diagram.

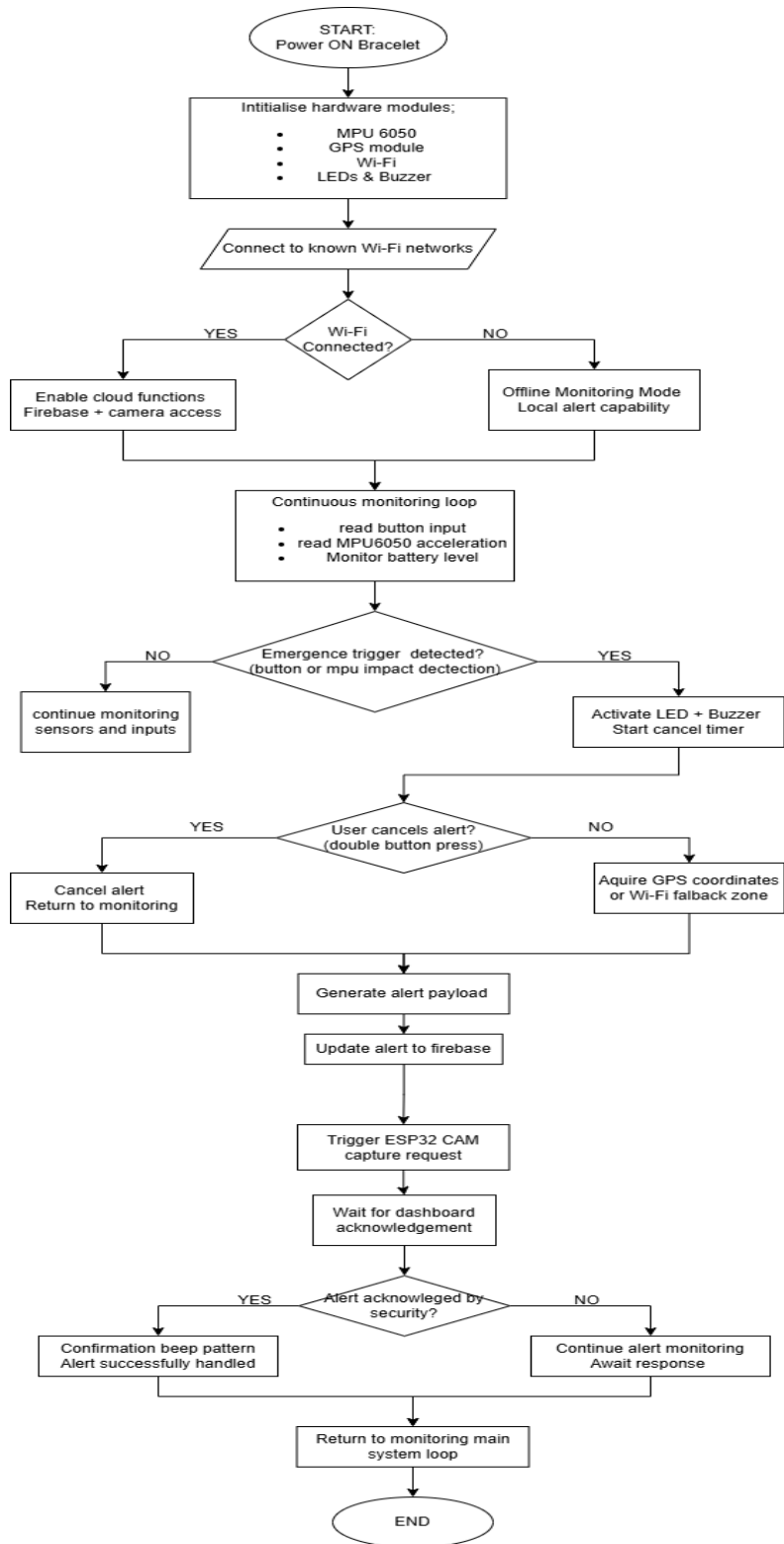


Figure3. 3 SOS Bracelet- Hardware and system logic block diagram

Here the MPU6050 accelerometer and gyroscope sensor continuously monitors wearer motion to detect abnormal patterns linked to distress, such as sudden impacts or prolonged inactivity. The ESP32 processes these readings locally and transmits them to the cloud for AI-assisted alert prioritization and emergency response.

a. Prototype Circuit Integration and hardware assembly

Due to funding and procurement delays, the wearable bracelet prototype was built on a DIY prototyping board instead of a custom PCB. This allowed faster assembly, testing, debugging and iterative improvements within the project timeline.

The hardware was assembled by interconnecting the ESP32 microcontroller, MPU6050 motion sensor, GPS module, power management circuitry, an active buzzer, battery, buttons and LEDs as well as peripheral interfaces on a compact prototyping platform. Connections were organized for stable communication, reliable sensor operation and efficient power distribution.

Although a fully fabricated PCB was initially planned for better miniaturization and packaging, the prototyping board implementation provided sufficient functionality for real-time system testing, emergency alert generation, wireless communication and AI-assisted monitoring. This approach also reduced lead times and enabled the project to move forward despite hardware procurement constraints.

Wiring was kept minimal and modular to reduce interference and simplify troubleshooting. The prototyping approach delivered full functionality for real-time alert generation, wireless communication and system testing while accommodating component changes during development. Figure 3.4 shows the assembled bracelet prototype with component layout and interconnections.

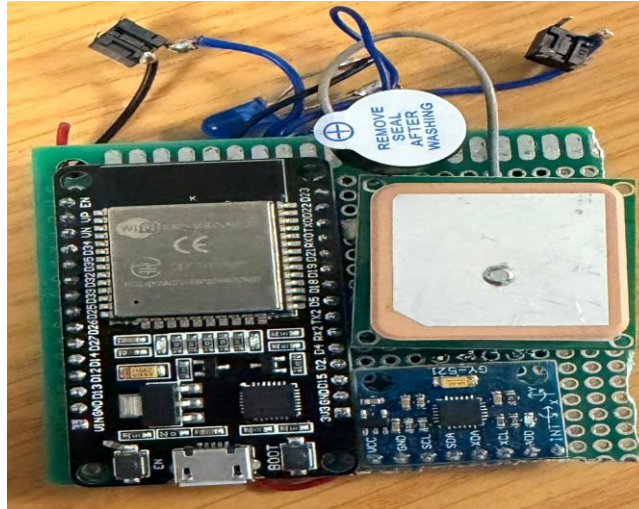


Figure3. 4 Assembled bracelet prototype on a DIY prototype board

b. 3D Printed Enclosure Design

A protective enclosure was designed to house the bracelet electronics, enhancing portability, durability, and wearing comfort. The enclosure was modeled using the SolidWorks CAD software and produced via 3D printing.

Key design considerations included:

1. Compact size with secure component protection and stability
2. Easy access to charging port and control interfaces
3. Adequate ventilation for heat dissipation during extended wear

Internal compartments securely held the prototyping board, battery, and sensors to prevent movement. Openings were provided for the SOS button, charging port, vibration feedback, and sensor interfaces.

3D printing allowed rapid prototyping and multiple design iterations to optimize fit, strength, and accessibility.

3.3.2 Clip-on Camera Integration

The clip-on body camera serves as an auxiliary evidence-capture module in the SafeCampus system. It provides visual context by capturing images when an emergency alert is triggered.

The camera integrates with the ESP32 platform via serial communication. An OV2640 module was chosen for its ESP32 camera libraries compatibility, compact size and reliable performance in embedded image capture.

a. Camera Logic and Operational Workflow

The camera logic was designed to operate conditionally in order to reduce unnecessary power consumption and storage usage. Instead of continuously recording, the camera activates only when a valid emergency condition is detected. This could be through either automatic distress detection based on abnormal motion patterns or manual button press trigger by the user.

Once triggered, the ESP32 initializes the camera, captures an image frame and transmits it to the cloud along with alert data. This event-driven design ensures relevant visual evidence is captured efficiently while maintaining low power consumption.

The operational workflow follows this sequence:

1. Motion sensor or manual trigger detects emergency.
2. ESP32 validates the event.
3. Camera initializes and captures image.
4. Image is temporarily stored in memory.
5. Image data is uploaded to the cloud with alert information.
6. Security personnel view the evidence via the dashboard.

Figure 3.5 illustrates the logical operation of the clip-on camera subsystem within the emergency response workflow.

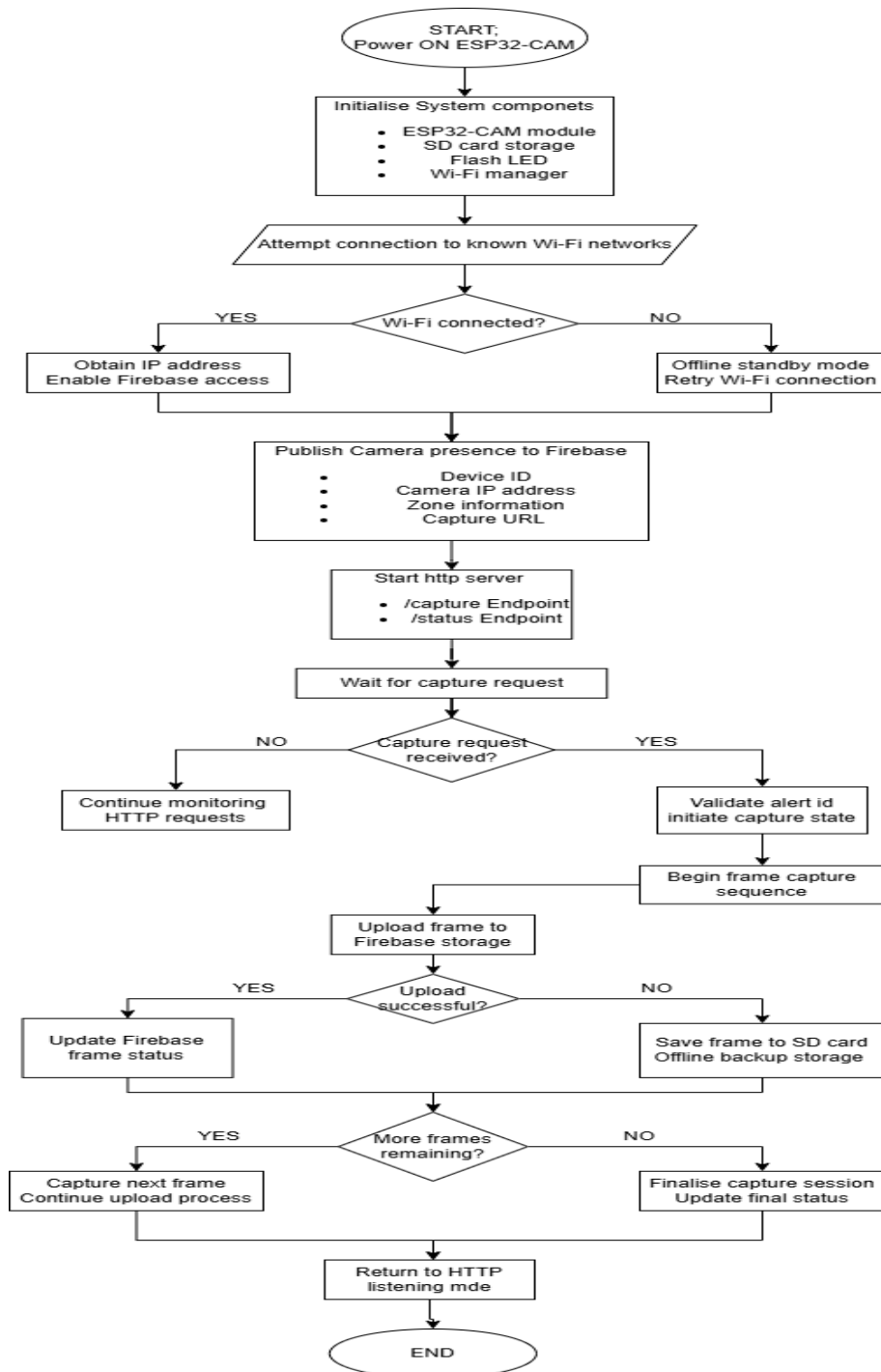


Figure3. 5 Clip-on Camera Operational Logic Flow

b. Image Capture and Transmission Process

Image capture and transmission were implemented using the ESP32 camera library within the wearable firmware. When an alert is triggered, the ESP32 initializes the OV2640 module and captures a compressed JPEG image to minimize size and bandwidth usage.

The image is then encoded and attached with metadata which includes timestamp, alert ID, GPS coordinates, and trigger type and then uploaded to the Firebase Storage database via Wi-Fi.

Stored images are linked in the camera_devices database node for quick retrieval. This enables security personnel to view visual evidence alongside alerts on the dashboard in near real time.

3.4 Software System Implementation

3.4.1 Cloud Database and Communication Layer

The backend of the proposed system was implemented using Firebase Realtime Database. Firebase was selected for its NoSQL structure and real-time synchronization capabilities. This cloud layer acts as the central communication hub of the system. It enables easy data exchange between the ESP32-based wearable device, AI threat prioritization model, campus security dashboard, Telegram patrol bot and student mobile application.

Firebase was particularly suitable for this project because it can instantly synchronize alert information across multiple interfaces. This is essential for time-sensitive campus emergency response applications.

a. Database Schema and Node Architecture

A hierarchical JSON tree structure was adopted to organize system data efficiently and support rapid read and write operations. The main database nodes include:

- i) An active_alerts node which stores real-time emergency alerts received from the bracelet. These alerts include GPS coordinates, student identification details, alert timestamp, trigger type, and AI-generated priority score.

- ii) A tipoffs node which stores anonymous reports submitted through the student mobile application. This allows students to communicate safety concerns without identity disclosure.
- iii) A campusPulse node which stores aggregated campus safety information. This includes safety status levels and general zone activity indicators displayed within the student app.
- iv) A history node which stores archived alert records after incident acknowledgment or resolution for historical tracking and analysis.
- v) A camera_devices node which stores metadata associated with the clip-on camera and uploaded image references used by the AI threat prioritization module.

This modular database organization improved maintainability while ensuring each subsystem accessed only the data relevant to its functionality.

b. Data Storage and Synchronization

A JSON-based data storage format was utilized to facilitate compatibility across the multiple software environments. This includes the ESP32 microcontroller and cam-module, Python-based Streamlit dashboard, Telegram bot scripts and Kivy mobile application. Firebase Realtime Database allowed automatic synchronization of newly generated alerts and status changes across all the connected modules. This therefore removed the need for frequently repeated manual refresh operations. It also ensured that updates made by one interface were immediately reflected in others.

c. Real-Time Communication Performance

Real-time communication was a very important design consideration because this was an emergency-response system. To support timely alert delivery, the security dashboard continuously monitored the active_alerts node for new alerts. The Telegram patrol bot retrieved alert data from Firebase and forwarded the critical incidents data to patrol officers. Alert acknowledgment updates made by security personnel were written back to Firebase and synchronized across the dashboard and student interfaces. This architecture allowed near real-time alert propagation and status synchronization. This improved overall system responsiveness during testing.

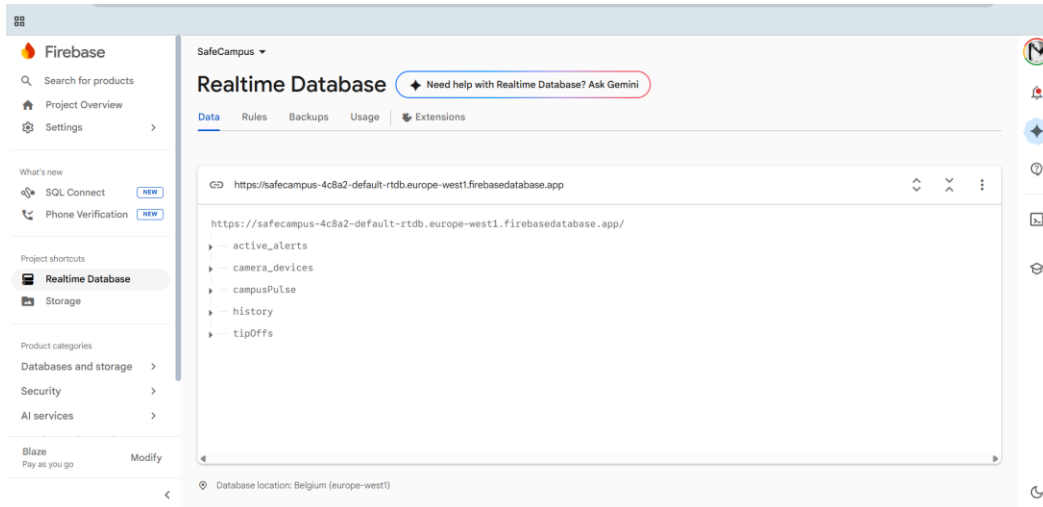


Figure3. 6 Firebase Realtime Database JSON Tree Structure

3.4.2 Security Dashboard Development

The security dashboard was developed as the primary and main monitoring interface for campus security personnel. It presents real-time emergency data collected from the bracelet system in an organized format. This enables immediate incident awareness and response.

a. Development Framework

The dashboard was implemented using Python and the Streamlit framework locally using Visual Studio Code. Streamlit was selected because of its simplicity in building an interactive web-based interface. It was also selected because of its compatibility with Python-based backend services. Integration with Firebase was achieved using the firebase-admin SDK. This allowed the dashboard to retrieve and display real-time alert information stored in the cloud database.

b. Functional Modules

The interface was structured into multiple modules to organize information according to operational priority. Firstly, we have live Campus Map which displays the geographical location of active incidents using GPS coordinates retrieved from the database. This enables security personnel to quickly identify the location of students requiring assistance.

Then we have Active Alerts module where we display ongoing emergency alerts received from the wearable bracelet. Alert entries include student identification details, timestamp, trigger type, location data, and AI-generated priority score. Next we have the Trends module which visualizes safety trends and incident patterns using weekly historical alert data. Graphical summaries help in identifying recurring incident patterns and frequently affected campus zones.

We also have History module where resolved alerts are stored. Alert records are retrieved from the history database node and displayed for incident review and record keeping. Lastly, we have Anonymous Tip-offs module which displays student-submitted anonymous reports received through the mobile application. This allows campus security to monitor non-emergency safety concerns.

c. User Interface Design Considerations

The user interface was designed to prioritize active alerts while keeping supporting information easily accessible. Clear visual hierarchy places ongoing emergencies as the most prominent element to ensure immediate attention.

A simple navigation bar allows security personnel to quickly switch between Active Alerts, History, Trends and Anonymous Tip-offs. This layout improves usability and helps reduce response time during critical situations.

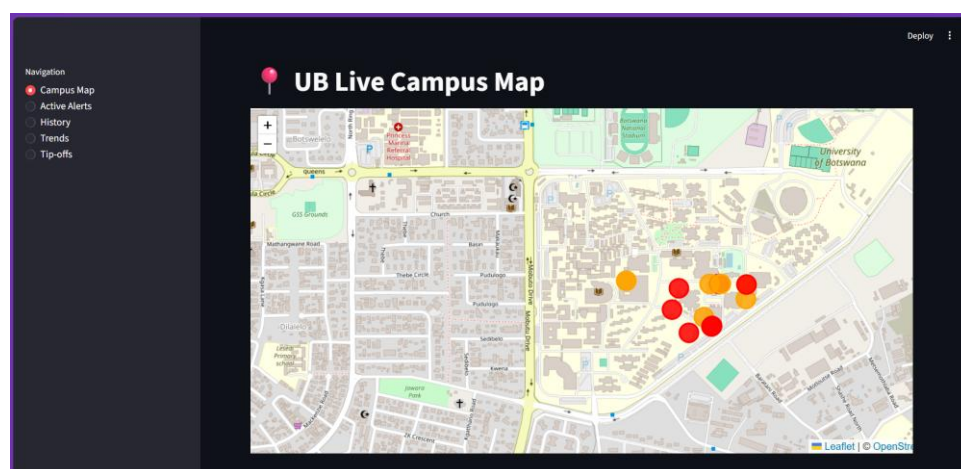


Figure3. 7 Security Dashboard Interface

3.4.3 Student Mobile Application Development

The student mobile application serves as the community layer component of the system. It was developed to help make the system more proactive than reactive. It also aides communication between students and campus security personnel while providing access to relevant campus safety information. The application complements the safety bracelet by offering additional safety features accessible directly from a mobile interface.

a. Development Framework

The mobile application was developed using the Kivy framework. It was selected for its ability to support Python-based cross-platform application development. Kivy was suitable for this project due to its compatibility with the Python ecosystem already used in other system components. This also includes Firebase integration and backend communication processes. The application was developed and tested locally using Visual Studio Code.

b. Functional Modules

The student mobile application consists of multiple modules designed to support different safety-related functions;

Firstly there is Campus Pulse. This module is a safety status system of the campus. It retrieves aggregated information from the campusPulse database node. The aggregated information compiled from the trends, the history and whether there is an active alert at the specific zone. It provides students with visibility into overall campus safety conditions at a glance.

Next we have Anonymous Tip-offs where students submit anonymous reports related to suspicious activities, hazards, or safety concerns. Submitted reports are stored in the tipoffs database node and made visible to campus security through the dashboard. This helps promote whistleblowing.

Then there is the trends section. It is a mirror of the trends at the security dashboard. It presents summarized weekly trends per zone. This allows students to remain informed about campus

activity. The warnings module displays anonymous safety warnings whenever an active alert has been reported so students may tread with care.

c. User Interface Design

The application interface was designed with usability and accessibility as primary considerations. A simple navigation structure was used to reduce complexity. It also ensures that critical features such as anonymous tip submission remain easily accessible. High-visibility buttons and organized module layouts were used to improve usability. This design approach supports efficient and easy user interaction.



Figure3. 8 Student Mobile Application Home Interface

3.4.4 Telegram Patrol Bot Integration

The Telegram Patrol Bot was developed to provide access to active alerts for campus security personnel while on patrol. Unlike the centralized dashboard, which is intended for stationary monitoring, the Telegram bot enables field personnel to receive and respond to emergency alerts directly from their mobile devices while on patrol. This integration improves system accessibility by extending emergency communication beyond the control dashboard.

a. Development Framework

The Telegram bot was implemented using the Telebot (pyTelegramBotAPI) library in Python and developed in Visual Studio Code. The bot was integrated with Firebase to retrieve alert data stored in the active_alerts node. When a new alert is generated by the bracelet, relevant information is retrieved and formatted into a Telegram message. This message is only viewed by designated security personnel.

b. Functional Features

The Telegram Patrol Bot allows the patrol officers to check for active alerts. Upon detection of active alerts, the bot automatically sends a message containing relevant incident information such as student identification details, timestamp and alert location to the patrol officer. GPS coordinates received from the bracelet are converted into a clickable Google Maps link, allowing security personnel to access the incident location directly from Telegram. Security personnel can acknowledge alerts through the Telegram interface. Once acknowledged, the updated alert status is written back to Firebase and synchronized across the security dashboard and other connected interfaces.

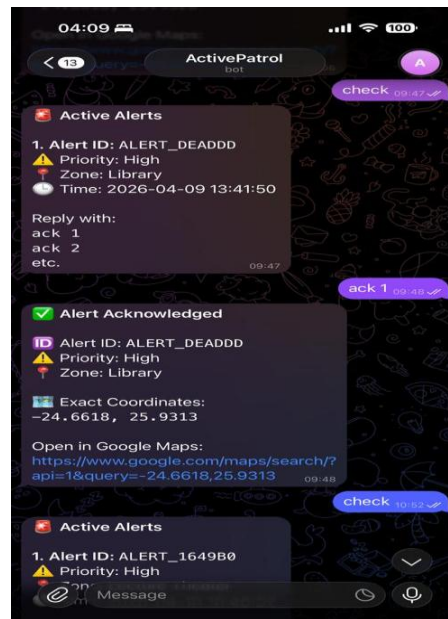


Figure3. 9 Telegram Patrol Bot

3.4.5 AI Threat Prioritization Model

The AI Threat Prioritization Model was developed to enhance emergency response efficiency by automatically analyzing incoming alerts and assigning priority levels based on image context and trigger conditions. It runs as a cloud-based system integrated with Firebase Realtime Database, enabling real-time decision support on the SafeCampus monitoring platform.

Unlike traditional wearable systems that depend only on manual activation, this model combines captured images with sensor-trigger data to help security personnel quickly identify and respond to potentially serious incidents.

a. AI System Architecture

The AI subsystem was implemented using Python and integrates two primary machine learning components:

- i. A Convolutional Neural Network (CNN) contextual scene classification model
- ii. A YOLOv8 object detection model for weapon detection

The CNN model analyzes environmental conditions and behavioral context from captured camera frames while the YOLO model performs object detection to identify potential weapons within the scene. The AI backend continuously monitors the Firebase `active_alerts` node for newly generated emergency events. Once an active alert is detected, the system retrieves uploaded image frames from Firebase Cloud Storage and begins automated threat analysis.

Figure 3.10. The figure is included to show how image frames and alert metadata are processed through the CNN and YOLO models before threat prioritization results are synchronized with the dashboard system.

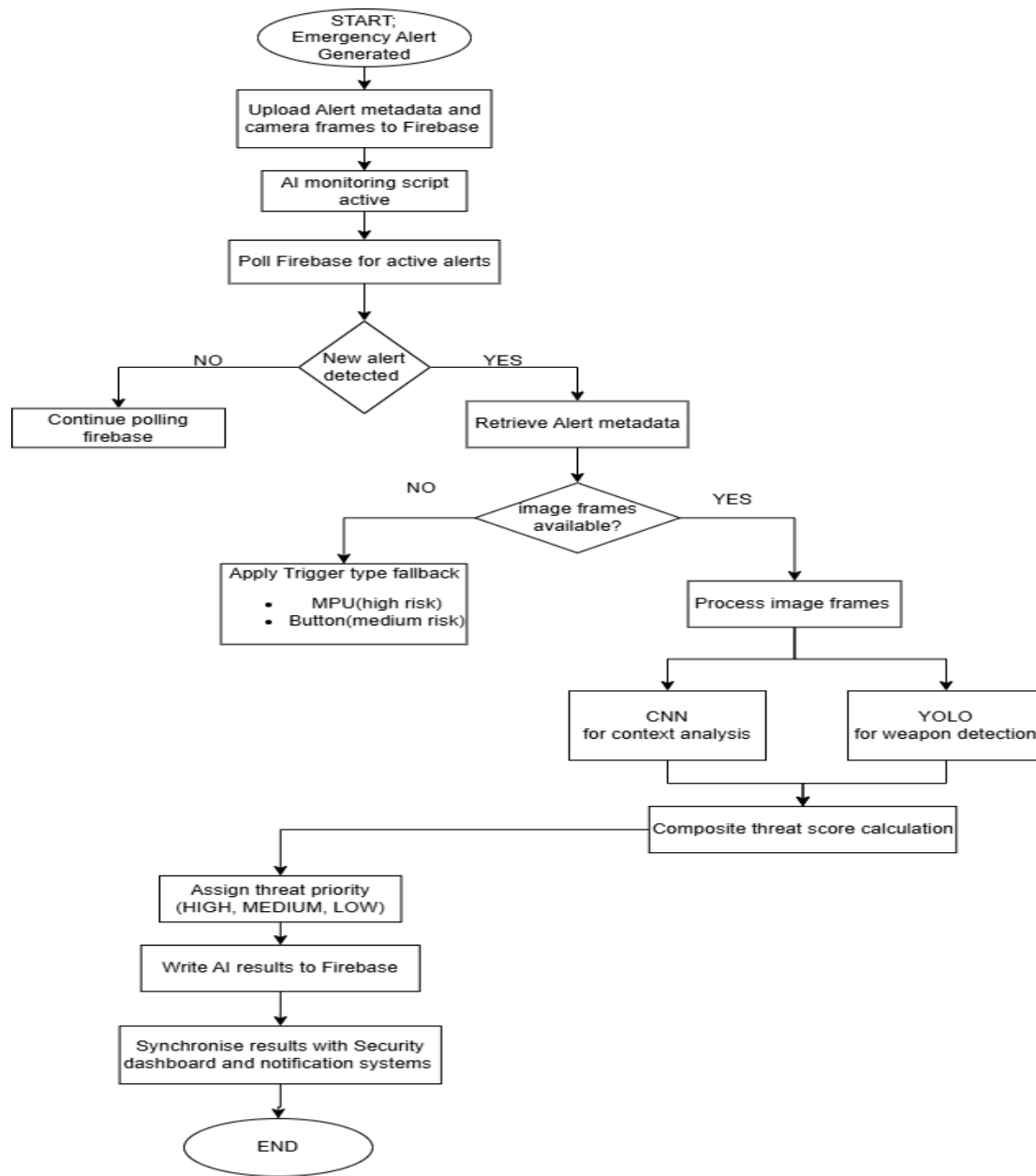


Figure3. 10 The AI processing workflow

b. Contextual Scene Classification Model

The contextual classification model classifies environmental and behavioral conditions from captured images to assess distress risk. Images are resized and preprocessed before being fed into the CNN model, which outputs probability scores for these scene categories:

- i. Crowd
- ii. Dark
- iii. Isolated
- iv. Normal
- v. Running

These outputs help estimate environmental risk. For instance, “Running” and “Isolated” indicate possible distress or pursuit, “Dark” suggests reduced visibility or unsafe conditions, while “Normal” lowers the threat weighting.

The system processes multiple frames per alert and applies exponential moving average smoothing to stabilize the classification results.

c. YOLO Weapon Detection Integration

To improve threat assessment accuracy, a YOLOv8 object detection model was integrated into the AI pipeline for weapon detection. The YOLO model analyzes uploaded image frames independently from the CNN classifier and generates confidence scores for detected weapon-like objects. A confidence threshold mechanism was implemented to minimize false detections:

- i. Minimum detection confidence threshold: 0.45
- ii. High-confidence override threshold: 0.70

When strong weapon detections occur across multiple frames the system increases the overall threat score and may override lower contextual scores generated by the CNN classifier. This integration improves prioritization accuracy for potentially violent incidents.

d. Threat Scoring and Priority Assignment

Threat prioritization is performed using a weighted scoring system combining the Contextual CNN classification outputs, YOLO weapon detection confidence and trigger source information from the bracelet system.

The scoring algorithm assigns weighted importance to different scene characteristics. Higher weights are assigned to weapon detections, running behavior or isolated environments. Lower or negative weights are assigned to normal scene classifications to reduce unnecessary false alarms. The system computes a final threat score ranging from 0 to 1 and assigns corresponding priority levels:

- i. LOW Priority
- ii. MEDIUM Priority
- iii. HIGH Priority

Additional prioritization logic was implemented for wearable-trigger conditions:

- i. MPU-triggered alerts automatically elevate incidents to HIGH priority because they may indicate physical assault or sudden impact events detected by the motion sensor
- ii. Button-triggered alerts without visual evidence default to MEDIUM priority until further data becomes available

This hybrid scoring approach improves reliability while maintaining fast response capability.

e. Cloud-Based Processing Workflow

The AI model runs entirely in the cloud instead of on the wearable device. This reduces power and processing demands on the bracelet while enabling more advanced image analysis. The processing workflow is as follows:

1. Emergency alert is sent from the bracelet to Firebase.
2. Captured image frames are uploaded to Firebase Cloud Storage.
3. The AI monitoring script detects the new alert.
4. Images are downloaded for processing.
5. CNN contextual analysis and YOLO weapon detection are performed.

6. Threat scores and priority levels are generated.
7. Results are written back to Firebase.
8. Updated information synchronizes to the security dashboard and Telegram bot.

The AI provides threat level, numerical score, smoothed frame results, weapon confidence values, and timestamps to help security personnel assess incident severity quickly.

f. Real-Time Alert Synchronization

After processing, the AI subsystem updates the corresponding Firebase alert node with the threat classification results. These updates automatically synchronize to the Streamlit security dashboard and Telegram patrol bot.

High-priority incidents are visually highlighted on the dashboard to ensure immediate attention. This real-time synchronization improves communication and reduces emergency response delays.

3.4.6 Hybrid Location Mapping System

To improve reliable location and user tracking during emergency situations, a hybrid location mapping system was implemented. This combines GPS-based positioning with Wi-Fi-based zone mapping.

The primary location method used in the system was GPS. The bracelet continuously attempts to obtain GPS coordinates through the GPS module when an alert is triggered. These coordinates are transmitted together with the rest of the alert data to the cloud database.

A geofencing script running within the backend system processes the received GPS coordinates by comparing them against predefined campus zones. The campus environment was modeled using a GeoJSON map containing polygon boundaries representing different university locations. When coordinates fall within a defined polygon, the system converts the raw latitude and longitude values into human-readable zone names. This allows security personnel to quickly identify the approximate incident location.

However in cases where GPS data is unavailable or unreliable, the system utilizes Wi-Fi-based fallback localization. This method estimates user location by identifying the connected Wi-Fi access point or router associated with the device. Different campus areas have distinct network access points. Therefore router associations can be mapped to approximate campus zones.

For prototype testing, predefined routers were used to emulate campus Wi-Fi infrastructure. These routers were manually assigned to specific zones. This allowed the system to simulate Wi-Fi-based localization behavior in a controlled environment.

The combination of GPS, geofencing, and Wi-Fi fallback improves system robustness by reducing dependence on a single location method. This hybrid approach increases the likelihood of obtaining usable location information during emergency situations, particularly in environments where GPS acquisition may be slow or faulty.

3.5 Testing And Validation

System testing and validation were conducted to assess the functional reliability, responsiveness of the campus safety system. The testing process covered alert triggering mechanisms, cloud synchronization, multi-platform communication, and AI threat prioritization.

3.5.1 Alert Trigger Testing

Alert trigger testing was done to verify if alerts could be successfully initiated through both manual and automatic mechanisms.

a. Manual Trigger Testing

The manual trigger mechanism was tested by activating the emergency push button integrated into the wearable bracelet. During each test the SOS button was pressed. The system was monitored to confirm generation of an emergency event. Location, timestamp, visual evidence and student identification data were checked for successful transmission to Firebase. This test validated the ability of the bracelet to generate emergency alerts through intentional user activation.

b. Automatic Trigger Testing

Automatic trigger testing was performed using the accelerometer-based shake detection mechanism. The bracelet was subjected to simulated vigorous movement conditions intended to represent distress or emergency motion. The system was monitored to check if acceleration threshold conditions were detected, and emergency alerts were automatically generated. The system was also checked to see if corresponding alert data was transmitted to the cloud database. This test validated the system's ability to detect emergency-like physical activity without manual user interaction.

3.5.2 Database Synchronization Testing

Database synchronization testing was performed to verify real-time consistency of data across connected components. The test procedures involved generating alerts and monitoring whether newly created records were correctly reflected in Firebase. It also checked if this synchronized across all relevant interfaces. This testing ensured that system modules maintained consistent data states during operation.

3.5.3 Communication Testing

Communication testing was conducted to verify successful transmission of emergency data between system interfaces. Emergency alerts were generated and monitored to confirm their appearance on the security dashboard. Acknowledgement was also checked to see if it would appear on the database. Telegram integration was tested by generating emergency alerts and verifying if the Telegram Patrol Bot could access them and the relevant data. Acknowledgement appearance on database was checked as well. The student mobile application was tested to verify the display of relevant data and submission of anonymous tipoffs. Feedback back to the bracelet from the database was also tested. This testing confirmed bidirectional communication between the user interfaces and backend database.

3.5.4 AI Prediction Testing

AI testing was conducted to validate the threat prioritization model integrated into the system. Test images captured through the clip-on camera module were uploaded to the cloud storage bucket and processed by the AI model. Successful image retrieval from database storage and execution of image analysis process was monitored. Generated threat scores were then checked for successful integration into the alert workflow, including display on the security dashboard and storage within Firebase alert records. This testing verified the functional integration of AI-based alert prioritization within the broader emergency response system.

3.6 Ethical Considerations

Ethical considerations focused on data privacy, user consent, system security, and responsible AI usage. Due to the sensitive nature of the system, it was designed to minimize unnecessary exposure of user information. Data is only transmitted to the cloud database when an alert is triggered. This reduces continuous monitoring and limits unnecessary collection of personal data.

To support user autonomy, the bracelet system incorporates a manual override mechanism. This allows users to cancel automatically triggered alerts in cases of false activation. This helps reduce unintended emergency dispatches and the user in control over system behavior.

As the system includes a clip-on camera and AI threat priority scorer, image handling was also considered from an ethical perspective. Camera data is intended solely for emergency verification and threat assessment purposes. Visual evidence is temporarily stored for processing and are not intended for long-term retention beyond operational necessity. Only authorised personnel can access to image data is restricted to authorized system interfaces only.

System security considerations were addressed through secure cloud-based data storage using Firebase. This was achieved with controlled access permissions implemented to reduce unauthorized database interactions. Only authorized modules interact with the database.

During testing and simulation, mock GPS coordinates, synthetic alerts, and controlled sample data were used where necessary to avoid unnecessary exposure of personal or sensitive information.

3.7 Limitations

The proposed system successfully demonstrates the integration of wearable hardware, cloud communication, AI prioritization and multi-platform monitoring interfaces. However several limitations were identified during development and testing.

A primary limitation is that the system remains at a prototype stage and has not yet undergone large-scale real-world deployment across an active campus environment. This means that long-term operational reliability under continuous real-world use could not be fully evaluated.

Although GPS functionality was implemented, location accuracy remains dependent on environmental conditions, signal quality, and module performance. GPS precision may be reduced in areas with signal obstruction or poor connectivity.

System communication performance is dependent on internet connectivity. Variations in network stability may introduce delays in alert transmission under live deployment conditions.

The AI priority scorer was tested using a limited dataset and controlled image inputs. This means performance may vary when exposed to real-world scenarios such as varying lighting conditions and image quality differences.

Testing was conducted within controlled development conditions. This means we could not fully replicate all real-world variables such as hardware wear, unpredictable user behavior, network congestion, or large numbers of simultaneous alerts.

In addition, the current system assumes access to system users and devices. Institutional deployment would require further security hardening and operational governance procedures. Despite these limitations, the developed prototype provides a functional proof of concept for an AI-enabled wearable campus bracelet system and establishes a strong foundation for future refinement, large-scale testing, and deployment.

CHAPTER FOUR

RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the results obtained from the implementation and testing of the proposed wearable AI-enabled bracelet system. It also provides an analysis of its functionality across the hardware and software components

4.2 Hardware Prototype Results

4.2.1 Wearable Bracelet results

The wearable bracelet prototype was tested to check its ability to detect emergency situations and communicate alerts to the cloud database. Testing focused on manual activation, automatic detection, user feedback mechanisms, and connectivity status indicators.

The results confirmed that the emergency button successfully generated alerts when pressed. Each activation initiated an emergency alert that was transmitted to the Firebase database in real time.

The automatic accelerometer-based mechanism successfully detected rapid or excessive movement patterns, which were used to simulate distress conditions. When the threshold was exceeded, an automatic alert was generated without manual input.

Upon triggering an alert, the system produced an initial buzzer sound indicating that an emergency alert was about to be sent. A 10-second cancellation window was provided, during which the user could cancel the alert by pressing the button. This feature successfully prevented unintended alerts during testing

Once an alert was successfully transmitted to the cloud database, the system provided feedback through a second buzzer sound and flicking LED indicator response to confirm successful alert submission

A double buzzer feedback mechanism was used to indicate that the alert had been acknowledged by security personnel. This helps put the user at ease as now they know help is on the way.

An LED indicator was used to display Wi-Fi connectivity status. When the device was connected to a network, the LED remained active, confirming successful network connectivity for alert transmission.

Overall, the wearable bracelet prototype successfully demonstrated reliable emergency alert generation, user feedback mechanisms, and cloud connectivity functionality, forming the foundation of the system's real-time alerting capability

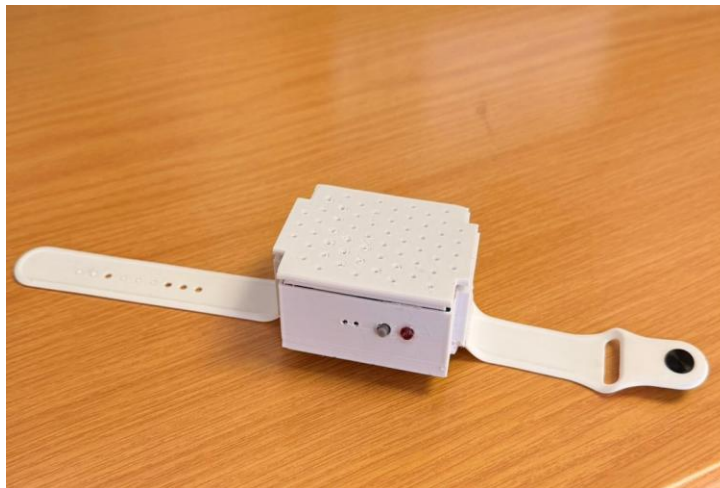


Figure 4. 1 Final Prototype of Bracelet

4.2.2 Clip-on Camera Results

The clip-on camera module was evaluated to determine its ability to capture and transmit visual data during emergency events triggered by the wearable bracelet. The module forms a key part of the system's evidence-gathering mechanism. It also supports the AI-based priority scoring process.

During testing, the clip-on camera successfully received trigger signals from the wearable bracelet upon activation of an emergency event.

Once triggered, the camera module was able to capture images with a delay of 3 seconds in between. It was then able to upload the visual data to the Firebase Cloud Storage bucket. The uploaded images were correctly linked to the corresponding alert entries within the active_alerts database node. This ensured proper association between visual evidence and emergency events.

The results confirmed successful integration between the hardware trigger system and the cloud storage pipeline. This enabled real-time transmission of visual evidence for further processing by the AI model for priority scoring.

Overall, the clip-on camera module successfully demonstrated its intended function of capturing and transmitting visual evidence during emergency conditions. This therefore enhances the system's situational awareness capabilities.

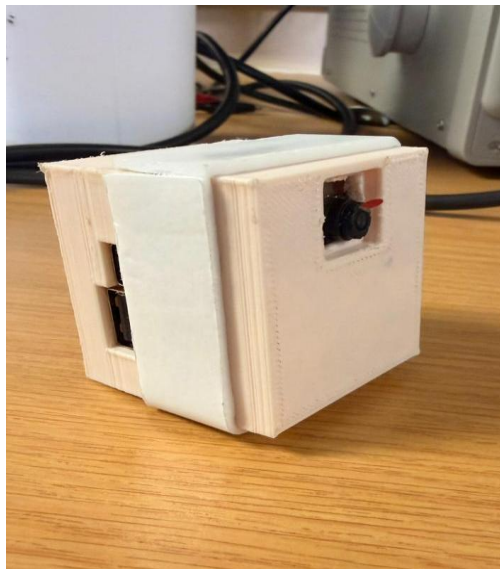


Figure 4. 2 Clip-on Camera prototype

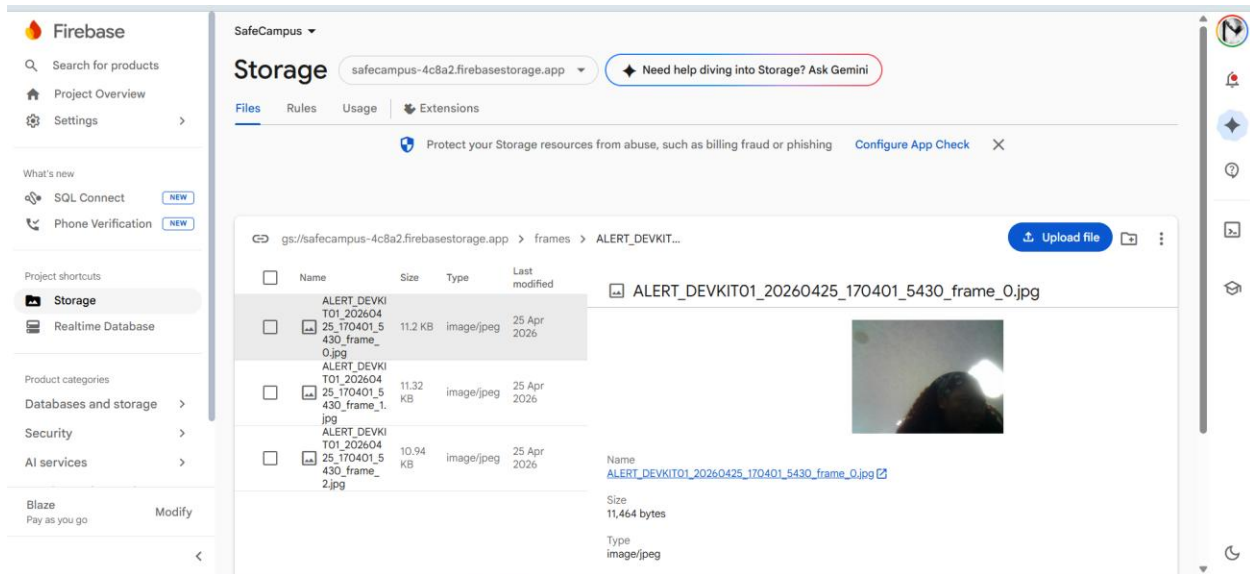


Figure 4. 3 Picture captured by Clip-on camera showing in the database storage bucket

4.3 End-to-End System Results

This section shows how every part of the system responds correctly from start to finish to a triggered alert.

4.3.1 Firebase logging

After an alert is triggered in the bracelet, the successful transmission is verified by the real-time creation of a new entry in the Firebase `active_alerts` node. This includes the bracelet's unique ID, location details, and a timestamp. This moment represents the critical transition from a physical distress signal to a digital data point, making the alert visible to the entire security network instantly.

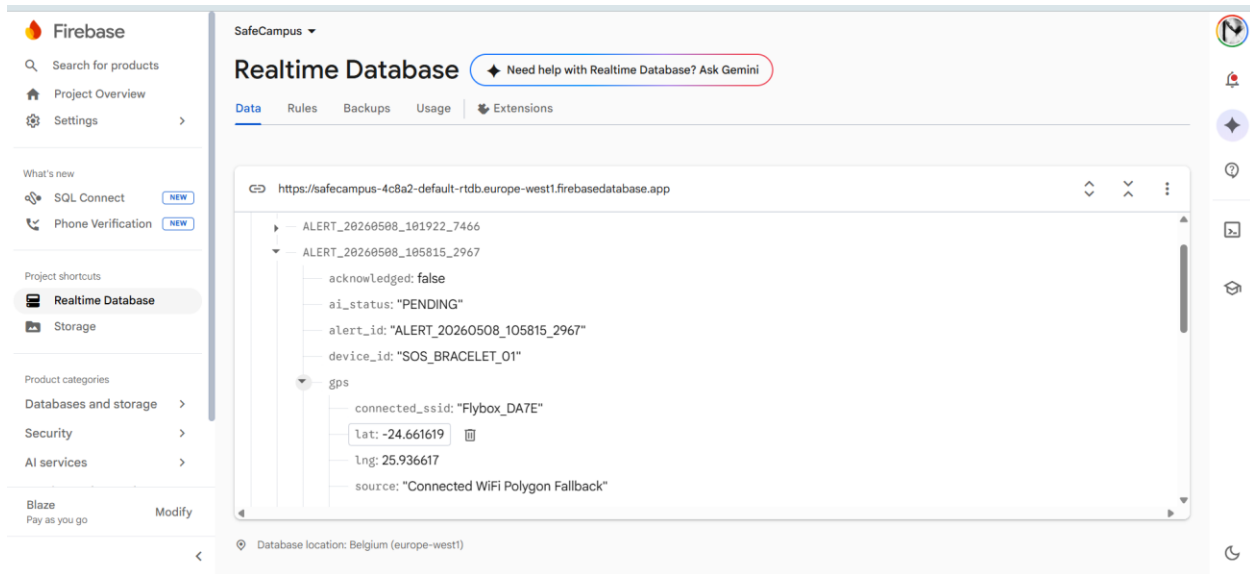


Figure 4. 4 Alert logged at the cloud database

4.3.2 Security Dashboard Response

Once the Firebase entry is logged, the Security Dashboard automatically updates through its persistent data listener. A new incident marker appears on the Live Campus Map, pinpointing the exact location of the student.

At the same time, the Active Alerts tab updates to the latest entry, displaying the student's details and the AI Priority Score. This score allows the operator to instantly distinguish between different levels of urgency. This stage confirms the successful bridge between the remote hardware and the central monitoring station. It ensures the emergency is visible to security personnel within seconds.

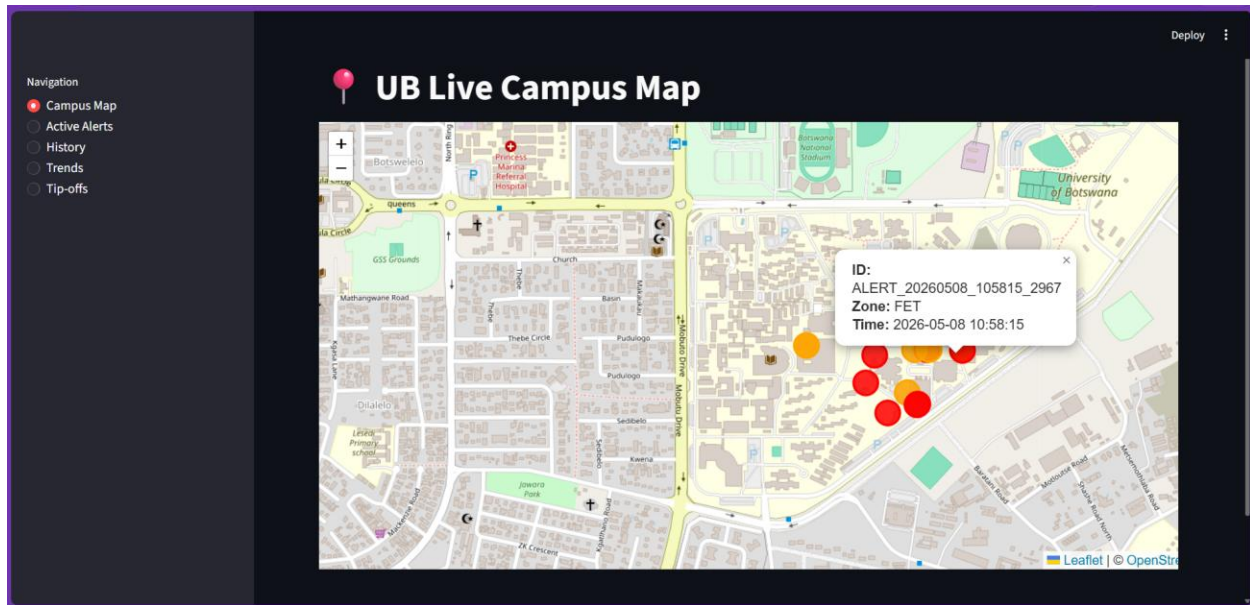


Figure 4. 5 Alert being shown in the security dashboard's campus map

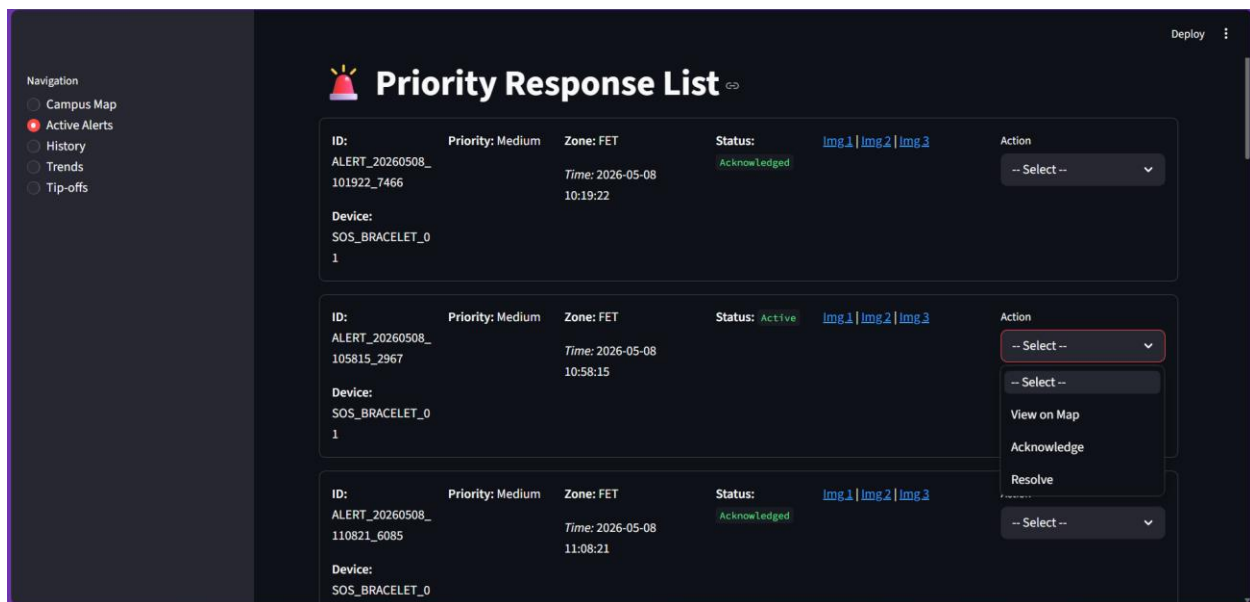


Figure 4. 6 Alert priority and more details being shown in the active alerts list

4.3.3 Telegram Patrol Bot Response

The Telegram Patrol Bot checks for active alerts and is able to access the active alerts. This alert contains the formatted incident details such priority, location and alert ID.

The interface also features an acknowledgment feature in which the officer types “ack” followed by the number of the alert they want to attend to. This action triggers a real-time status update back to the Firebase active_alerts node. This generates a Google Maps GPS link. By clicking this link, the officer’s mobile device automatically launches its navigation app, displaying the student’s live coordinates on a satellite or street-view map.

This closes the loop between the field response and the central command. This stage ensures that the responding officer has immediate, turn-by-turn guidance to the exact location of the emergency

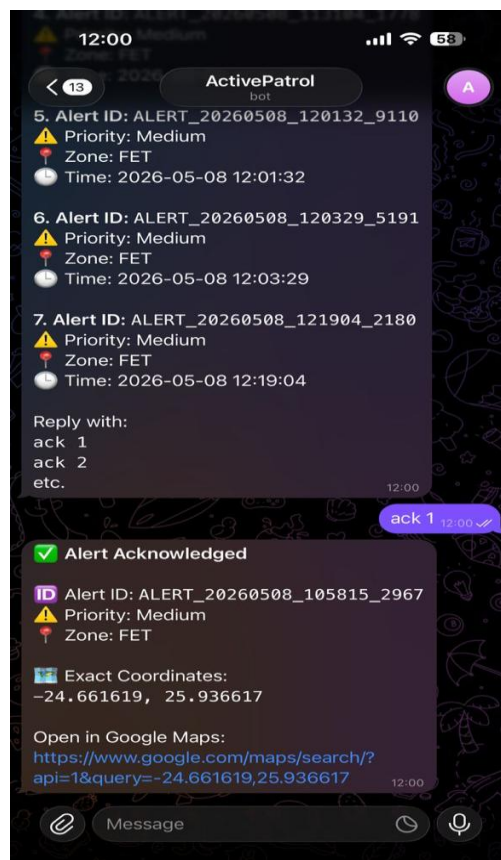


Figure 4. 7 Alert being acknowledged in telegram bot

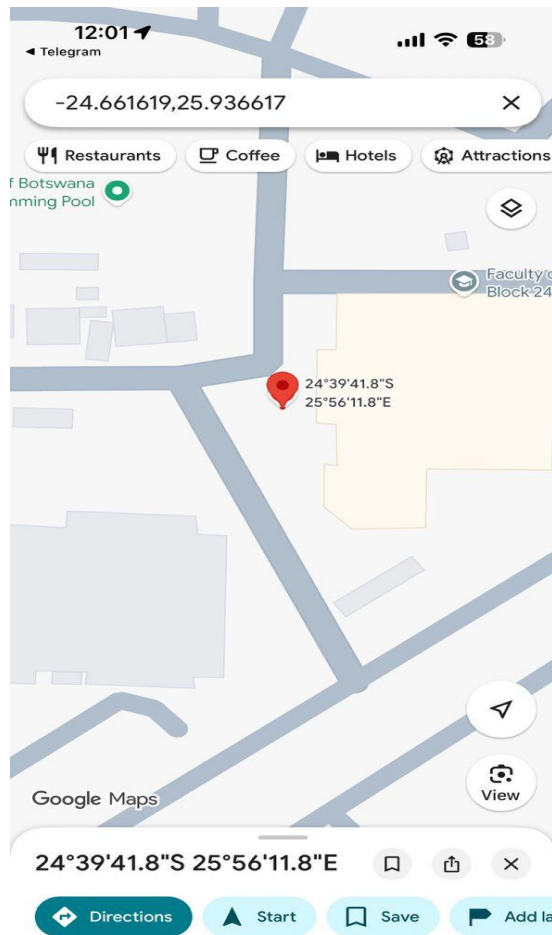


Figure 4. 8 Google Maps link opened from the telegram bot

4.3.4 Student App Response

The student mobile app (SafeCampus) completes the feedback loop by reflecting the current security state of the campus. It turns the system from a reactive one to a proactive one. Campus Pulse uses color coordinated blocks to show the safety status of different zones in the Campus. Red is for warning, Yellow for caution and Green for safe. Campus Pulse uses data from the trends , history and active alerts to determine the safety of the particular zone at that specific time the student is checking the app. This helps student make well-informed decisions regarding their movement around campus as they will be avoiding areas that are in the red for example. The

Warnings module simultaneously populates with localized notifications, informing nearby students of the ongoing incident to ensure they avoid the affected area or proceed with caution.

The Anonymous Tip-off module allows students to submit text reporting suspicious activity that may be in relation to the ongoing active alert. Once submitted, these tipoffs appear instantly in the Firebase tipoffs node and on to the security dashboard. Beyond receiving information, this stage confirms the app's bidirectional functionality. This ensures the student ecosystem remains both protected by and a contributor to the overall safety network.

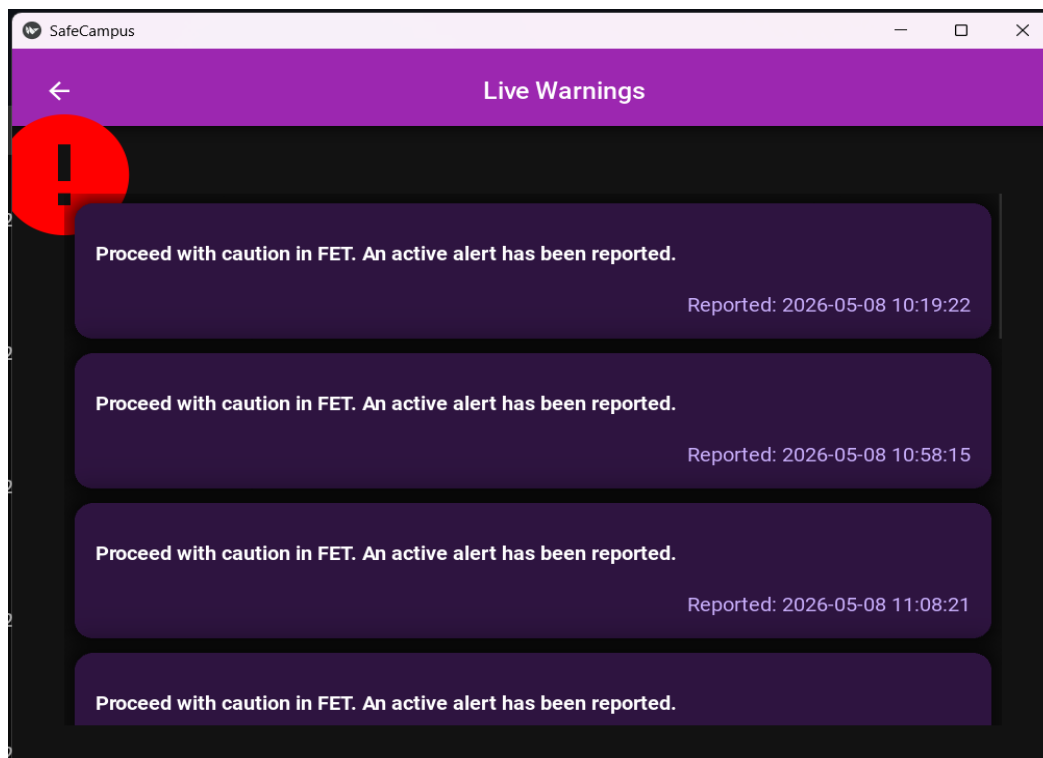


Figure 4. 9 Warnings showing in student app

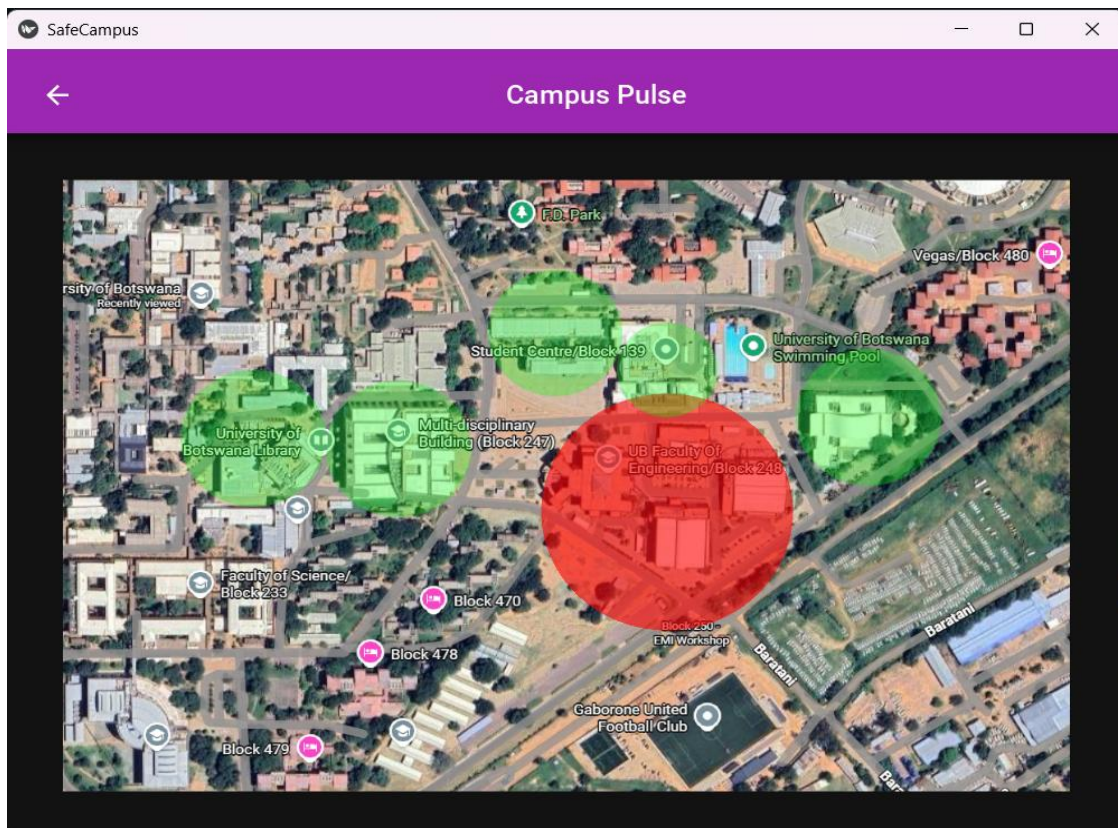


Figure 4. 10 Campus Pulse in student app showing current safety status of campus

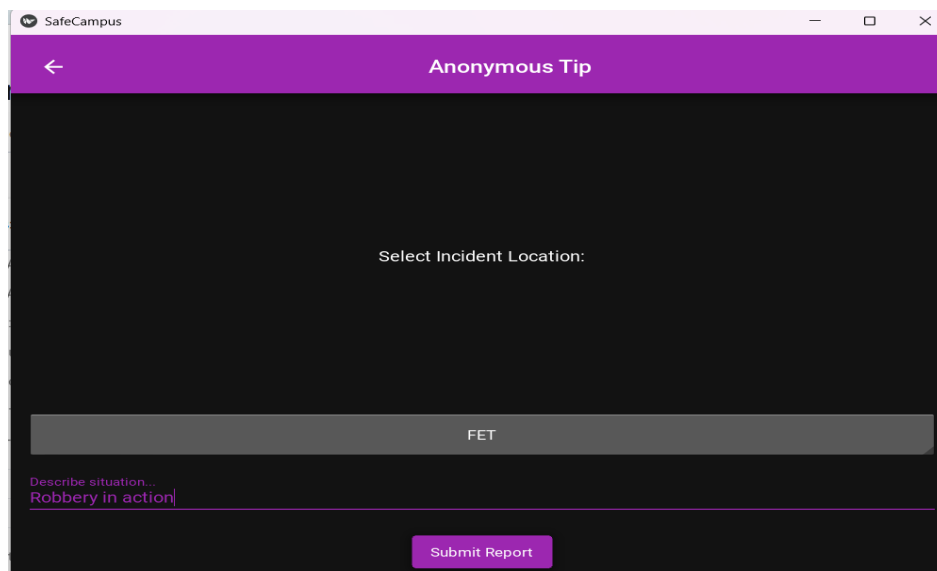


Figure 4. 11 Anonymous tipoff being sent from student app

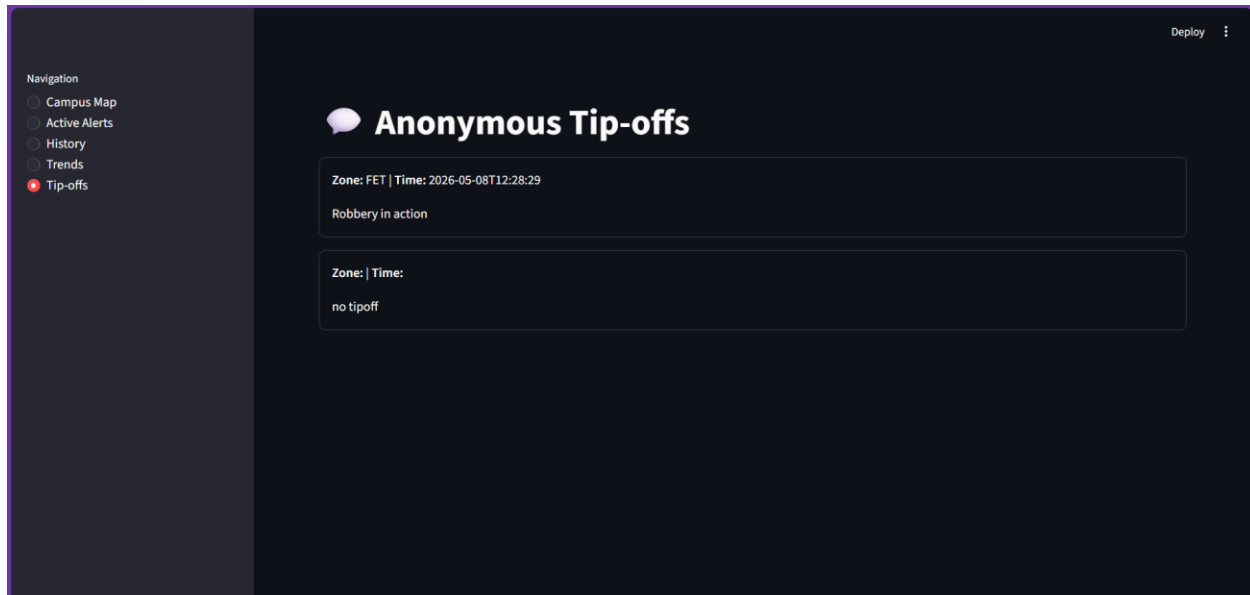


Figure 4. 12 Same anonymous tipoff from student app being displayed in security dashboard

4.3.5 Incident History and Data Management

The final stage of the operational cycle is the migration of data for long-term storage and auditing. Once an alert incident is marked as "Resolved" at the security dashboard, the system automatically triggers a transfer protocol. This process moves the data from the active_alerts node to the history node in Firebase. This process keeps the live monitoring feed uncluttered while ensuring a permanent record of the event is preserved. The data from the history node is also sorted into weekly trends . These trends are displayed at the security dashboard and mirrored at the student app.

To manage this data, the dashboard features a Search function. This allows security personnel to query past incidents by student ID, date, or priority level. Furthermore, a CSV Download utility is implemented. This enables the security department to export the entire history log into a spreadsheet format. This capability transforms raw incident data into an organized knowledge base for administrative reporting and campus safety trend analysis.

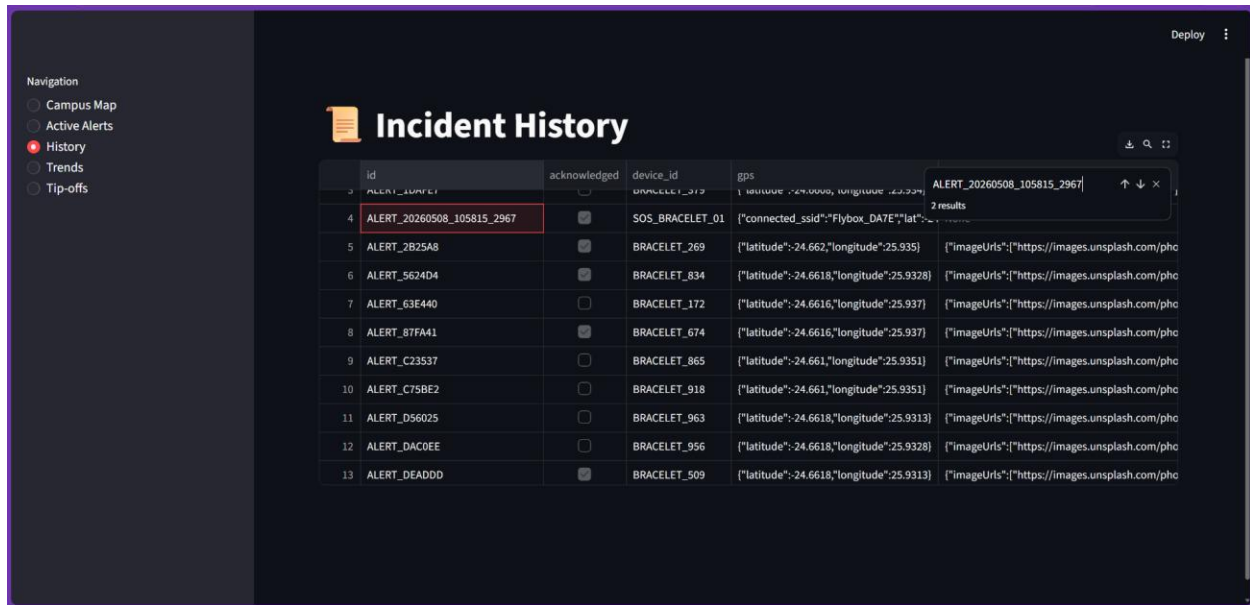


Figure 4. 13 Resolved alert is moved to history section

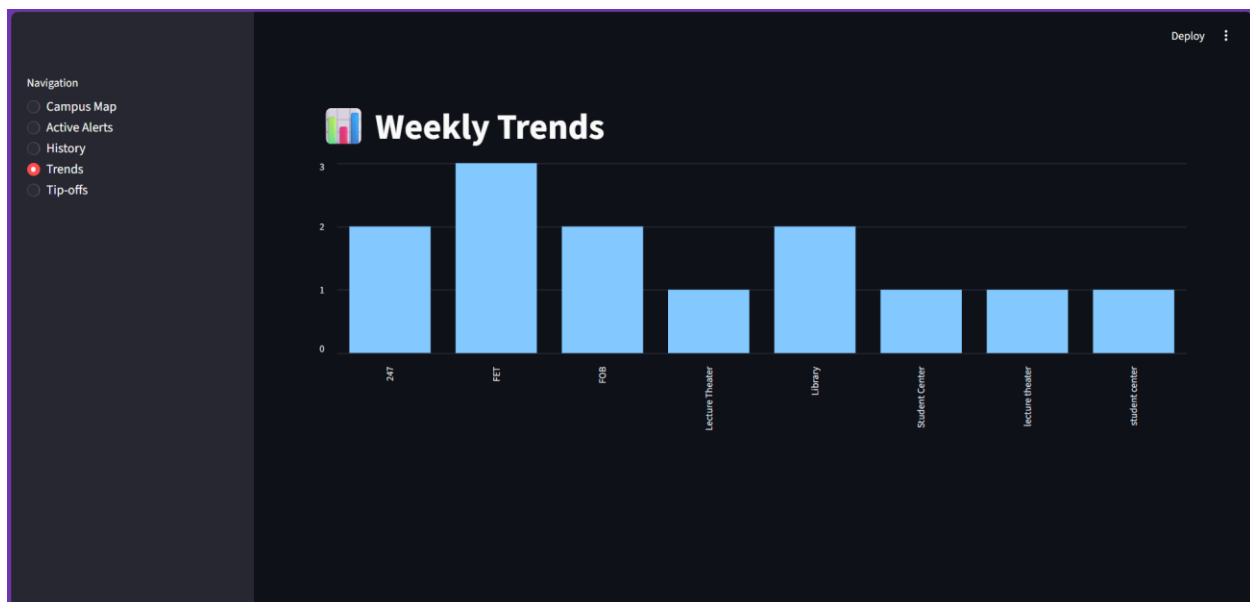


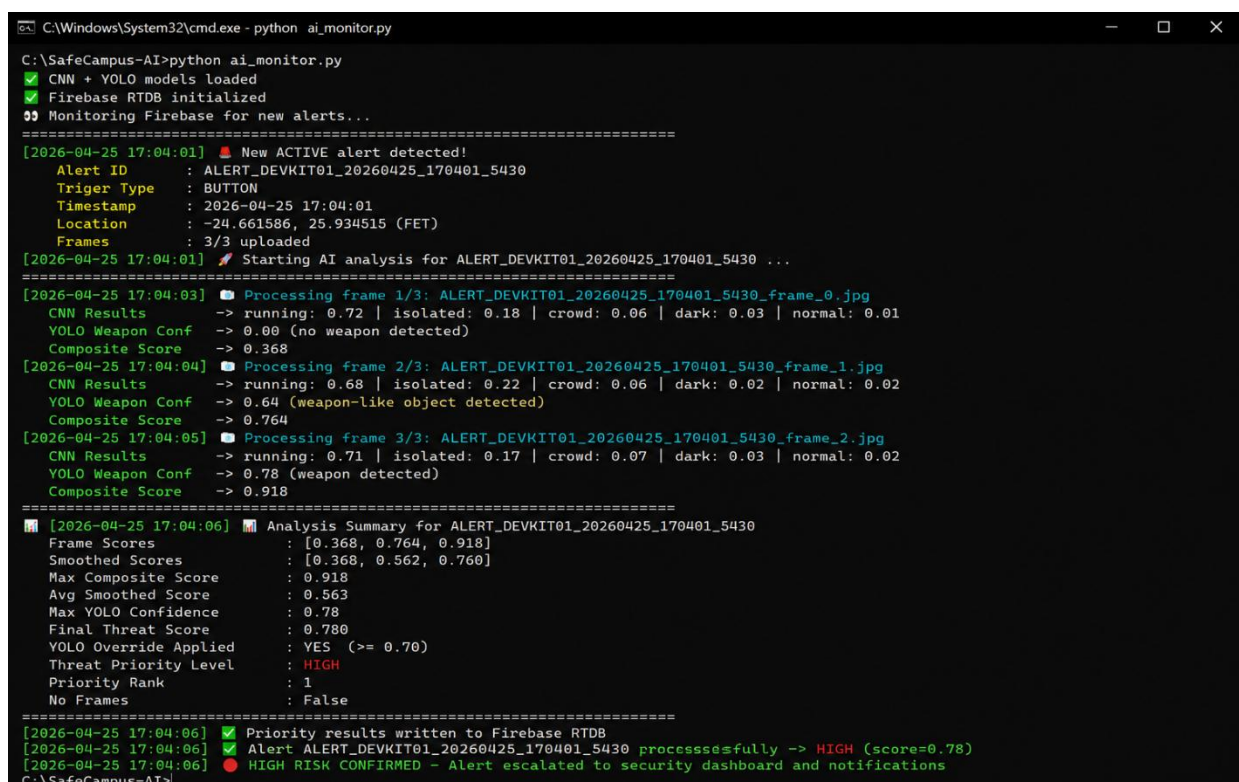
Figure 4. 14 Weekly trends of all alerts per zone

4.3.6 AI Priority Scoring Results

The AI monitoring system successfully processed the uploaded alert frames using both the CNN contextual analysis model and the YOLO weapon detection model. It computed a composite threat score based on environmental context and weapon confidence values before assigning the final priority level.

During testing, the model analyzed the frames from the generated alert and produced a HIGH priority classification due to strong threat indicators across multiple frames. The workflow included frame retrieval from Firebase Cloud Storage, contextual analysis, weapon detection, weighted risk calculation, and exponential smoothing.

Figure 4.15 shows the command-line output from the AI system after processing the emergency alert and assigning the final priority level.



```
C:\Windows\System32\cmd.exe - python ai_monitor.py
C:\SafeCampus-AI>python ai_monitor.py
[2026-04-25 17:04:01] CNN + YOLO models loaded
[2026-04-25 17:04:01] Firebase RTDB initialized
[2026-04-25 17:04:01] Monitoring Firebase for new alerts...
[2026-04-25 17:04:01] New ACTIVE alert detected!
Alert ID      : ALERT_DEVKIT01_20260425_170401_5430
Trigger Type  : BUTTON
Timestamp     : 2026-04-25 17:04:01
Location      : -24.661586, 25.934515 (FET)
Frames        : 3/3 uploaded
[2026-04-25 17:04:01] Starting AI analysis for ALERT_DEVKIT01_20260425_170401_5430 ...
[2026-04-25 17:04:03] Processing frame 1/3: ALERT_DEVKIT01_20260425_170401_5430_frame_0.jpg
CNN Results   -> running: 0.72 | isolated: 0.18 | crowd: 0.06 | dark: 0.03 | normal: 0.01
YOLO Weapon Conf -> 0.00 (no weapon detected)
Composite Score -> 0.368
[2026-04-25 17:04:04] Processing frame 2/3: ALERT_DEVKIT01_20260425_170401_5430_frame_1.jpg
CNN Results   -> running: 0.68 | isolated: 0.22 | crowd: 0.06 | dark: 0.02 | normal: 0.02
YOLO Weapon Conf -> 0.64 (weapon-like object detected)
Composite Score -> 0.764
[2026-04-25 17:04:05] Processing frame 3/3: ALERT_DEVKIT01_20260425_170401_5430_frame_2.jpg
CNN Results   -> running: 0.71 | isolated: 0.17 | crowd: 0.07 | dark: 0.03 | normal: 0.02
YOLO Weapon Conf -> 0.78 (weapon detected)
Composite Score -> 0.918
[2026-04-25 17:04:06] Analysis Summary for ALERT_DEVKIT01_20260425_170401_5430
Frame Scores      : [0.368, 0.764, 0.918]
Smoothed Scores   : [0.368, 0.562, 0.760]
Max Composite Score : 0.918
Avg Smoothed Score : 0.563
Max YOLO Confidence : 0.78
Final Threat Score : 0.780
YOLO Override Applied : YES (>= 0.70)
Threat Priority Level : HIGH
Priority Rank       : 1
No Frames          : False
[2026-04-25 17:04:06] Priority results written to Firebase RTDB
[2026-04-25 17:04:06] Alert ALERT_DEVKIT01_20260425_170401_5430 processed fully -> HIGH (score=0.78)
[2026-04-25 17:04:06] HIGH RISK CONFIRMED - Alert escalated to security dashboard and notifications
C:\SafeCampus-AI>
```

Figure 4. 15 AI priority scoring results

4.4 Discussion

4.4.1 System Integration and Functional Performance

The results obtained demonstrate that the proposed On-Campus Wearable AI-Enabled Security Bracelet System was successfully implemented as an integrated campus safety ecosystem. It combined wearable hardware, cloud communication, mobile interfaces, and AI threat prioritization. The bracelet prototype successfully supported both manual and automatic alert triggering mechanisms. Manual activation was achieved through the emergency push-button, while automatic triggering was achieved through motion-based distress detection. This confirmed that the hardware could respond to both deliberate and involuntary emergency scenarios.

Once triggered, alerts were successfully transmitted from the bracelet to the cloud database. Here they were immediately synchronized across the security dashboard, Telegram bot, and student mobile application. This confirms successful end-to-end communication between the hardware and software components.

The successful synchronization of alerts across multiple interfaces demonstrates the feasibility of integrating wearable IoT devices into campus emergency response systems. This aligns with existing research on smart safety systems, which emphasizes the importance of interconnected real-time communication between sensing devices, cloud platforms, and monitoring interfaces for rapid emergency response.

Additionally, the successful transfer of alert data from detection to visualization validates the modular system architecture proposed in this study. The ability of all components to operate together as one system indicates that the developed prototype met its primary functional objectives.

4.4.2 Real-Time Synchronization and Latency Performance

System responsiveness is very important for emergency response systems. During testing, the prototype demonstrated near real-time alert synchronization. The alerts were appearing on the

dashboard approximately 5 to 8 seconds after bracelet activation under stable network conditions. In all successful tests, total latency remained below 10 seconds.

This relatively low delay indicates efficient communication between the ESP32 microcontroller, cloud database, and developed user interfaces. This response time is actually considered acceptable for a prototype-level campus safety system. This is because it allows security personnel to receive and assess alerts within a practical time window following incident detection.

The real-time synchronization observed between the bracelet, dashboard, mobile application, and Telegram bot suggests that the implemented cloud communication layer effectively supports low-latency data exchange. This is consistent with IoT-based safety research, which identifies cloud synchronization and lightweight wireless communication as important factors in reducing response delays.

However, system performance remains dependent on internet stability. Testing was conducted under reliable Wi-Fi conditions. Latency may increase under weak or unstable network environments. This highlights the importance of incorporating redundant communication methods in future system versions.

4.4.3 Hybrid Location Mapping Performance

One of the key contributions of this project was the implementation of a hybrid location mapping system combining GPS and Wi-Fi-based localization.

Primary location tracking was using GPS coordinates. However, because GPS signal acquisition may be delayed or unavailable in certain environments, a fallback Wi-Fi localization method was implemented. This approach mapped user location based on connected Wi-Fi access points, taking advantage of the distributed campus router infrastructure.

The Wi-Fi association enabled approximate zone detection even when GPS coordinates were unavailable because different campus zones are served by distinct router identifiers. During testing, this method successfully provided location context by identifying predefined Wi-Fi zones.

This hybrid approach improved the system robustness by reducing dependence on a single localization method. Similar studies in smart campus and indoor localization systems have shown that combining GPS with Wi-Fi-based methods improves positioning reliability, especially indoors.

For simulation purposes, predefined routers were used to emulate campus access points. While this simplified testing, it also demonstrates the scalability of the approach, as future deployment could map actual institutional Wi-Fi infrastructure for more accurate localization.

The successful implementation of hybrid mapping suggests that combining multiple localization strategies is practical and beneficial for campus safety applications.

4.4.4 Platform Performance and User Interface Functionality

The software platforms developed in this project performed according to design expectations. The security dashboard successfully displayed active alerts with their priorities, live campus map, historical resolved alerts, weekly trends, and anonymous student tip-offs. The movement of resolved alerts into the history section confirms that the system supports alert lifecycle management. This allows incidents to be tracked from detection through resolution.

The student mobile application successfully displayed Campus Pulse safety analytics, warning notifications, weekly trends, and anonymous reporting functionality. The synchronization of trend data between the dashboard and mobile application demonstrates consistency across user-facing interfaces.

The Telegram bot also functioned as intended by retrieving active alerts and corresponding GPS coordinates. This provided security personnel with a lightweight alternative interface for rapid access to emergency information.

Furthermore, the AI threat prioritization model successfully retrieved uploaded images from cloud storage, analyzed incident data, and generated corresponding priority scores. This demonstrates the feasibility of incorporating intelligent risk assessment into campus safety workflows.

The successful performance of these software components supports existing research suggesting that modern safety systems benefit from centralized monitoring and multi-platform accessibility. Overall, the platform results indicate that the software ecosystem effectively complements the wearable hardware, creating a coordinated response framework rather than an isolated alert device.

4.4.5 Prototype Limitations and Future Improvements

Although the prototype achieved its primary objectives, several limitations were identified during implementation. First, the wearable bracelet prototype was physically larger than intended. This reduced comfort and wearability. The camera module and protective casing also contributed to overall bulkiness which limited the practicality of long-term daily use.

Due to material availability constraints, the bracelet was fabricated using rigid plastic rather than more flexible wearable materials. This may have been sufficient for prototyping but this reduced ergonomic quality and user comfort.

Another limitation was the absence of GSM/GPRS communication. The current prototype relies primarily on Wi-Fi-based communication for alert transmission. Although the hybrid localization system improves reliability, the lack of GSM backup means the system remains vulnerable in scenarios where Wi-Fi connectivity is unavailable.

Future versions should integrate GSM communication to enable mobile network-based alert transmission in the absence of Wi-Fi. This would improve system reliability and reduce dependence on institutional network infrastructure.

Further improvements should also focus on hardware miniaturization through custom PCB design, smaller component packaging, and more compact camera integration. Reducing overall device size would significantly improve usability and increase adoption likelihood among students.

Despite these limitations, the current prototype successfully demonstrates the feasibility of an integrated wearable campus safety system combining emergency alerting, intelligent prioritization, hybrid localization, and real-time multi-platform communication.

CHAPTER FIVE

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This project successfully developed the SafeCampus system which is an IoT-based wearable campus safety solution designed to improve emergency response and student safety. The system integrated a wearable SOS bracelet, clip-on camera, cloud database, security dashboard, student mobile application, Telegram bot, and AI threat prioritization model into a unified safety ecosystem.

The bracelet successfully supported both manual and automatic alert triggering. Alerts were being transmitted and synchronized across all connected interfaces. The security dashboard displayed active alerts, alert history, trends, and anonymous tip-offs. The student mobile application provided safety analytics through Campus Pulse, warning notifications, and anonymous reporting functionality. The Telegram bot successfully retrieved active alerts and location data.

The implementation of hybrid location mapping using GPS and Wi-Fi improved localization reliability. It provided fallback location identification when GPS data was unavailable. The AI threat prioritization model successfully analyzed uploaded images and generated alert priority scores. Latency testing showed near real-time synchronization, with alerts appearing across interfaces within approximately 5 to 8 seconds under stable network conditions.

Although hardware limitations such as prototype size, bulky camera casing, rigid materials, and lack of GSM backup were identified, the system successfully demonstrated the feasibility of an integrated wearable campus safety solution. Overall, the project achieved its objective of developing a functional prototype for real-time campus emergency monitoring and response.

5.2 Recommendations for Future Study

Future improvements should focus on reducing the size of the bracelet and clip-on camera to improve comfort and wearability. More flexible materials should also be used to enhance the ergonomic quality of the device.

GSM/GPRS communication should be integrated as a backup communication method to improve reliability in the absence of Wi-Fi connectivity.

Future implementations should also use actual campus Wi-Fi infrastructure to improve localization accuracy and further refine the hybrid location mapping system. WiFi fingerprinting can be used for more accuracy.

Additionally, the AI threat prioritization model can be improved using larger datasets to enhance scoring accuracy.

Finally, real-world deployment and large-scale testing are recommended to evaluate long-term usability, system performance, and practical effectiveness in campus environments.

REFERENCES

- [1] G. Matsapa, "Campus safety SOS bracelet," *EEB 444: Electronics Experimental Design*, University of Botswana, Gaborone, Botswana, 2025.
- [2] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Sensors*, vol. 23, no. 5, pp. 2821–2840, Mar. 2023. [Online]. Available: <https://www.mdpi.com/1424-8220/23/5/2821>
- [3] J. Doe and A. Smith, "Campus security challenges in higher education: A review," *J. Campus Safety*, vol. 10, no. 2, pp. 45–60, Feb. 2024.
- [4] L. Chen et al., "Real-time location tracking for wearable safety devices," *IEEE Internet Things J.*, vol. 10, no. 3, pp. 2345–2356, Feb. 2023.
- [5] B. K. Rai, S. Kumar, and V. Kumar, "AI based human safety smart bracelet," *Int. J. Innov. Res. Comput. Commun. Eng.*, vol. 12, no. 3, pp. 1–8, Mar. 2024. [Online]. Available: https://www.researchgate.net/publication/394548247_AI_BASED_HUMAN_SAFETY_SMART_BRACELET
- [6] A. K. Tripathy et al., "IoT based women safety band using Blynk application," *Int. J. Res. Pract. Sci.*, vol. 6, no. 4, pp. 1–7, Apr. 2025. [Online]. Available: <https://ijrpr.com/uploads/V6ISSUE4/IJRPR41640.pdf>
- [7] N. T. Nguyen et al., "Wearable safety solutions for women: A case study," *J. Women's Health Technol.*, vol. 8, no. 2, pp. 67–75, Apr. 2024.
- [8] R. K. Gupta et al., "Wearable IoT devices for emergency response systems," *IEEE Trans. Consum. Electron.*, vol. 69, no. 4, pp. 123–130, Nov. 2023.
- [9] S. Lee and H. Kim, "AI-driven wearable safety systems: A survey," *IEEE Access*, vol. 11, pp. 56789–56800, Jun. 2023.

- [10] K. Zovko and L. Šerić, "IoT and health monitoring wearable devices as enabling technologies for sustainable enhancement of life quality in smart environments," *J. Clean. Prod.*, vol. 389, pp. 136–148, Feb. 2023.
- [11] P. Radanliev, "Privacy, ethics, transparency, and accountability in AI systems for wearable devices," *Front. Digit. Health*, vol. 7, pp. 1431246, May 2025.
- [12] Google, "Add Firebase to your Flutter app," *Firebase Docs*, 2025. [Online]. Available: <https://firebase.google.com/docs/flutter/setup>
- [13] Cirkuit Designer, "ESP32-based portable GPS tracker with GSM communication and motion sensing," *Cirkuit Docs*, 2025. [Online]. Available: <https://docs.cirkuitdesigner.com/project/published/2883f967-38f8-465c-817f-d20eb78008f2/esp32-based-portable-gps-tracker-with-gsm-communication-and-motion-sensing>
- [14] S. Jethani, P. Motwani, S. Mantri, and U. Pujeri, "Women's safety wearables design using an IoT-based framework technology," in *ICT: Innovation and Computing. ICTCS 2023*, A. Joshi, M. Mahmud, R. G. Ragel, and S. Karthik, Eds. Singapore: Springer, 2024, pp. 89–102. doi: 10.1007/978-981-99-9486-1_8.

APPENDICES

APPENDIX A-1: GANTT CHART

SAFE CAMPUS: ON-CAMPUS WEARABLE AI-ENABLED SECURITY BRACELET SYSTEM

GANTT CHART – PROJECT KEY STAGES AND TIMELINE

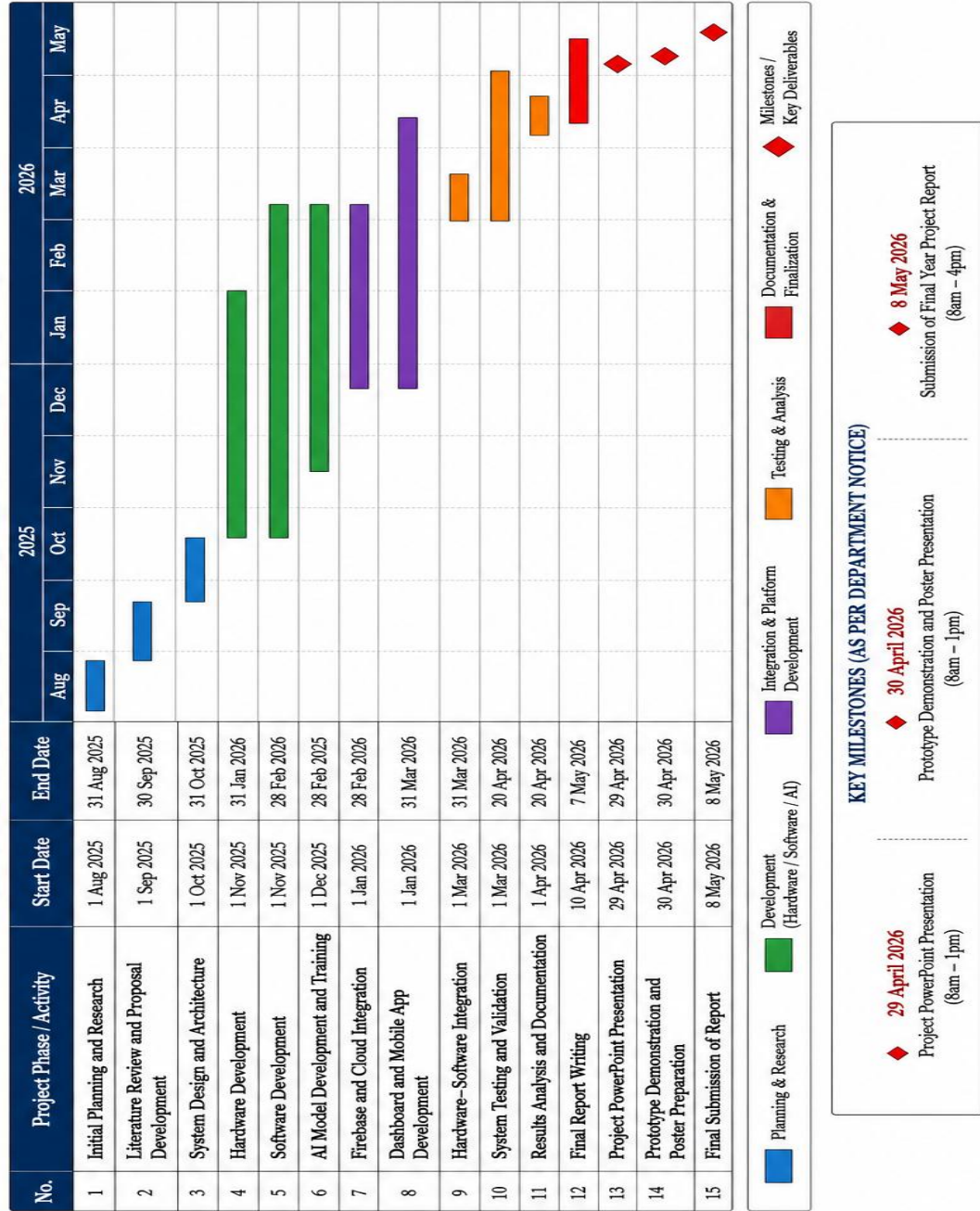


Figure A - 1 Gantt chart showing project key stages and timeline

APPENDIX A-2: BUDGET

Table 0.1 List of components required and budget

Component	Description	Quantity	Cost (BWP)	Source	Justification
ESP 32 Devkit WROOM		1	600	Local store	Core hub microcontroller; Supports Wi-Fi for cloud upload and body cam http request, low-power modes. Easily sourced
OV2640 Mini + ESP32 Cam Module	2MP camera sensor (1600x1200, 3-5V, compact clip-on for chest mounting).	1	380	Local store	Body cam accessory for first-person evidence; triggered by bracelet via Wi-Fi (SOS/motion), uploads photos to cloud. Low power (~50mA on-demand)
MPU-6050 Accelerometer	6-axis gyro/accelerometer (~3.6mA, 3-5V).	1	215	Local Store	Sole sensor for motion detection and stillness warnings. Low power, Arduino-compatible; sends data to cloud to bracelet for alert trigger.

NEO-6M GPS Module	GPS module (3-5V, ~20mA standby).	1	330	Local Store	Sends gps coordinates to cloud for dashboard mapping
LiPo Battery (3.7V, 500mAh)	Compact lithium-polymer battery (~30g).	1	108	Local Store	Powers bracelet hub for 4-6 hours, lightweight for wearability. Supports duty cycling for motion checks and cloud uploads.
TP4056 Charging Module	USB-C LiPo charger with protection (1A, 5V).	1	97	Takealot	Enables quick USB-C recharging.
Buzzer (5V)	Low-power active buzzer (~10mA).	1	27	Local shops	Audible alert for manual triggers or stillness warnings as well as acknowledgement feedback; low power for immediate feedback.
Micro SD Card (32GB)	For local media storage with auto-erase.	1	85	Local shops	Temporary storage for trigger-based photos (~100MB/day max); 48-hour auto-erase script before cloud upload

					prevents overflow/ privacy issues.
Firestore Credits (Basic Tier)	Free tier for storage/processing (up to 1GB/month, 10GB transfer).	1	135 - rated for premium if needed	Firestore (Google)	Main hub for secure cloud storage of emergency data (alerts, media) and AI processing; free tier suffices for prototype testing, with optional upgrade for scalability.
Resistors (10kΩ, 1kΩ)	For pull-up/pull-down and sensor circuits.	10	20	Local shops	Standard circuit protection; supports MPU-6050 and body cam interfacing.
Capacitors (10μF, 100nF)	For power stabilization and noise filtering.	10	27	Local shops	Ensures stable operation for ESP 32 devkit and sensors during cloud uploads.
Breadboard (mini)	Small prototyping board (400 points).	1	41	local shops	For initial testing of bracelet hub and body cam BLE pairing.
Jumper Wires	Female-to-female, male-to-female (20cm).	40	100	local shops	Flexible connections for breadboard prototyping.

3D Printing Filament (PLA)	1kg PLA for enclosure (~100g needed).	1	350 (pro-rated)	Local lab	For custom bracelet and body cam enclosures via university 3D printer; ensures discreet, durable design.
Push Button	Tactile switch for manual testing/reset.	2	25	Local shops	Manual distress trigger or response to stillness warnings; simple override for false positives.
Miscellaneous (Solder, Wires, PCB)	Soldering kit, hookup wires, small PCB for final assembly.	1 set	250	Local shops	For robust final connections between bracelet and body cam, including cloud upload testing.

Total Cost: P2,790(excluding 380~600 BWP for faulty or short circuited componets replacements)

APPENDIX B: PYTHON CODES

APPENDIX B-1: Dashboard Active Alert Retrieval and Priority Classification

```
def get_active_alerts():
    data = db.child("active_alerts").get().val()
    alerts = []
    if data:
        for alert_id, details in data.items():
            gps = details.get("gps", {})
```



```

raw_priority = details.get("priority", {}).get("final", "Medium")
trigger_val = str(details.get("trigger", "button")).lower()
# Convert numeric priorities into readable labels
if isinstance(raw_priority, (int, float)) or (
    isinstance(raw_priority, str) and raw_priority.isdigit()
):
    p_num = int(raw_priority)
    if p_num == 0:
        final_priority = "Low"
    elif p_num == 1:
        final_priority = "High"
    else:
        final_priority = "Medium"
else:
    final_priority = str(raw_priority)
# Force automatic alerts to high priority
if "button" not in trigger_val:
    final_priority = "High"
raw_lat = gps.get("lat")
raw_lon = gps.get("lng")
try:
    lat = float(raw_lat) if raw_lat not in [None, ""] else -24.6615
    lon = float(raw_lon) if raw_lon not in [None, ""] else 25.9325
except:

```

```

lat, lon = -24.6615, 25.9325

alerts.append({
    "id": alert_id,
    "device_id": details.get("device_id", "Unknown Device"),
    "priority": final_priority,
    "zone": gps.get("zone", "Unknown Zone"),
    "coords": [lat, lon],
    "status": "Acknowledged" if details.get("acknowledged") else "Active",
    "sent_time": details.get("timestamp", "Unknown")
})

return alerts

```

APPENDIX B-2: Student Mobile Application Real-Time Data Synchronization

```

def fetch_firebase_data(self, dt):
    try:
        alerts = db.child("active_alerts").get().val()
        dash_ids = self.root.get_screen("dashboard").ids
        warning_screen = self.root.get_screen("warnings")
        active_alerts = []
        if alerts:
            for alert_id, alert_data in alerts.items():
                if not isinstance(alert_data, dict):
                    continue
                resolved = str(alert_data.get("resolved", "false")).lower()
                if resolved in ["false", "no", "0"]:

```

```

        active_alerts.append((alert_id, alert_data))

        # Trigger notification for new alerts

        if alert_id not in self.known_alert_ids:

            self.known_alert_ids.add(alert_id)

            if self.alert_sound:

                self.alert_sound.play()

        # Update warning badge and warnings page

        dash_ids.warning_badge.opacity = 1 if len(active_alerts) > 0 else 0

        dash_ids.warning_badge.text = str(len(active_alerts))

        warning_screen.update_warnings(active_alerts)

        # Update trends data

        history_data = db.child("history").get().val()

        self.root.get_screen("trends").update_graph(history_data)

        # Update Campus Pulse safety zones

        pulse = db.child("campusPulse").get().val()

        if pulse:

            self.root.get_screen("campus_pulse").update_status(pulse)

    except Exception as e:

        print(f'Sync Error: {e}')

```

APPENDIX B-3: Telegram Bot Active Alert Retrieval

```

@bot.message_handler(func=lambda msg: msg.text and msg.text.lower().strip() == "check")
def check_alerts(message):

    try:

        ref = db.reference("active_alerts")

```

```

alerts = ref.get()

if not alerts:

    bot.reply_to(message, "No active alerts at the moment.")

    return

active_session_alerts.clear()

response = "Active Alerts\n\n"

alert_number = 1

for alert_id, data in alerts.items():

    if data.get("acknowledged") is True:

        continue

    active_session_alerts[str(alert_number)] = alert_id

    priority = get_formatted_priority(data)

    gps = data.get("gps", {})

    zone = gps.get("zone", "Unknown")

    timestamp = data.get("timestamp", "No timestamp")

    response += (

        f"{alert_number}. Alert ID: {alert_id}\n"

        f"Priority: {priority}\n"

        f"Zone: {zone}\n"

        f"Time: {timestamp}\n\n"

    )

    alert_number += 1

bot.send_message(message.chat.id, response)

except Exception as e:

```

```
bot.reply_to(message, f'Failed to access alerts: {e}')
```

APPENDIX B-4: Hybrid Location Mapping and Geofencing Logic

```
def load_geofences(filename):  
    with open(filename, 'r') as f:  
        data = json.load(f)  
  
    geofences = []  
  
    for feature in data['features']:  
        polygon = shape(feature['geometry'])  
        name = feature['properties'].get('zone_name', 'Unknown Zone')  
        geofences.append({"name": name, "polygon": polygon})  
  
    return geofences  
  
campus_zones = load_geofences('campus_zones.geojson')  
  
def get_zone_name(lat, lon):  
    point = Point(lon, lat)  
  
    for zone in campus_zones:  
        if zone['polygon'].contains(point):  
            return zone['name']  
  
    return "Outside Campus"  
  
def process_alert(alert_id, details):  
    if not details or "gps" not in details:  
        return  
  
    gps = details.get("gps", {})  
    lat = gps.get("latitude")  
    lon = gps.get("longitude")
```

```
if lat and lon:
```

```
    new_zone = get_zone_name(lat, lon)
```

```
    db.child("active_alerts").child(alert_id).update({"zone": new_zone})
```

APPENDIX B-5: AI Threat Detection and Threat Scoring Code Snippets

```
# =====
```

```
# ALERT PROCESSING
```

```
# =====
```

```
def process_alert(
```

```
    frames: list,
```

```
    trigger_type: str = "BUTTON"
```

```
) -> dict:
```

```
    scores = []
```

```
    for frame in frames
```

```
        cnn_result = process_image_with_retry(frame)
```

```
        yolo_conf = detect_weapon_yolo(frame)
```

```
        score = compute_frame_score(
```

```
            cnn_result,
```

```
            yolo_conf
```

```
        )
```

```
        scores.append(score)
```

```
    final_score = max(scores)
```

```
    if final_score < 0.2:
```

```
        level = "LOW"
```

```
    elif final_score < 0.7:
```

```
        level = "MEDIUM"
```

```
    else:
```

```
        level = "HIGH"
```

```
    # MPU-triggered alerts automatically HIGH
```

```

if trigger_type == "MPU":
    level = "HIGH"
return {
    "final_score": final_score,
    "level": level
}

```

APPENDIX C: ARDUINO IDE (.ino) CODES

APPENDIX C-1: ESP32-CAM Image Capture and Cloud Upload Code Snippet

```

// =====
// SD CARD FALLBACK STORAGE
// =====

void saveToSD(

    camera_fb_t* fb,
    String filename
) {
if (!sdAvailable) return;
File f = SD.open(
    "/frames/" + filename,
    FILE_WRITE
);if (!f) {
    Serial.println("SD write failed");
    return;
}
f.write(fb->buf, fb->len);
f.close();
Serial.println(
    "Saved to SD: " + filename

```

```

    );
}
// =====
// IMAGE CAPTURE + CLOUD TRANSMISSION
// =====

void captureAndUpload(String alertID) {
    Serial.println(
        "Starting capture: " + alertID
    );
    for (int i = 0;
        i < FRAMES_PER_ALERT;
        i++) {
        digitalWrite(FLASH_LED, HIGH);
        delay(60);
        camera_fb_t* fb =
            esp_camera_fb_get();
        digitalWrite(FLASH_LED, LOW);
        if (!fb) {
            Serial.println(
                "Frame capture failed"
            );
            continue;
        }
        currentAlert.captured++;
        String filename =
            alertID +
            "_frame_" +
            String(i) + ".jpg";
        bool ok = false;
        // Upload to Firebase

```



```

if (WiFi.status() == WL_CONNECTED) {
    currentAlert.state = "UPLOADING";
    ok = uploadToFirebase(
        fb,
        filename,
        alertID
    );
}
// SD card backup fallback
if (ok) {
    currentAlert.uploaded++;
} else {
    saveToSD(fb, filename);
}
esp_camera_fb_return(fb);
updateFrameStatus(
    alertID,
    currentAlert.uploaded,
    FRAMES_PER_ALERT
);
delay(CAPTURE_INTERVAL_MS);
}
// Final upload state
if (currentAlert.uploaded ==
    FRAMES_PER_ALERT) {
    currentAlert.state = "DONE";
} else if (currentAlert.uploaded > 0) {
    currentAlert.state = "PARTIAL";
} else {
    currentAlert.state = "FAILED";
}

```

```
}  
}
```

APPENDIX C.2: ESP32 DevKit Emergency Trigger Detection and Alert Workflow

```
// =====  
// BUTTON EMERGENCY TRIGGER (Double-press cancellation supported within 5 seconds)  
// =====  
  
void handleButton() {  
    static bool lastState = HIGH;  
  
    static unsigned long pressStart = 0;  
  
    bool current = digitalRead(BUTTON_PIN);  
  
    // Detect button press  
  
    if (lastState == HIGH && current == LOW) {  
        pressStart = millis();  
    }  
  
    // Detect button release  
  
    if (lastState == LOW && current == HIGH) {  
        unsigned long duration = millis() - pressStart;  
  
        // Long press enters deep sleep  
  
        if (duration > 3000) {  
            goToSleep();  
  
            return;  
        }  
  
        // Trigger SOS alert
```

```

if (millis() - lastTriggerTime > TRIGGER_COOLDOWN_MS) {
    alertPending = true;
    alertStartTime = millis();
    pendingSource = "BUTTON";
    digitalWrite(LED_PIN, HIGH);
    beep(2, 100, 300);
    Serial.println(
        "BUTTON alert triggered"
    );
}
}

lastState = current;
}

// =====
// MPU6050 IMPACT DETECTION (Detects repeated high acceleration spikes)
// =====

void checkMPU() {
    int16_t ax, ay, az;
    mpu.getAcceleration(&ax, &ay, &az);
    float g = sqrt(
        pow(ax / 16384.0f, 2) +
        pow(ay / 16384.0f, 2) +
        pow(az / 16384.0f, 2)
    );
}

```

```

// Detect strong acceleration event
if (g > 2.2f) {
    if (spikeCount == 0)
        firstSpikeTime = millis();
    spikeCount++;
}

// Confirm multiple spikes within 1 second
if (spikeCount > 0 &&
    millis() - firstSpikeTime > 1000) {
    if (spikeCount >= 2 &&
        !alertPending &&
        millis() - lastTriggerTime >
        TRIGGER_COOLDOWN_MS) {
        alertPending = true;
        alertStartTime = millis();
        pendingSource = "MPU";
        digitalWrite(LED_PIN, HIGH);
        beep(2, 100, 300);
        Serial.println(
            "MPU alert triggered"
        );
    }
    spikeCount = 0;
}

```

```

}

// =====

// HANDLE PENDING ALERT

// Core SOS workflow:

// 1. Resolve GPS/Wi-Fi location

// 2. Generate alert payload

// 3. Upload alert to Firebase

// 4. Trigger ESP32-CAM image capture

// =====

void handlePendingAlert() {
    if (!alertPending) return;
    if (millis() - alertStartTime > CANCEL_WINDOW_MS) {
        // Resolve best available location
        LocationResult loc = getLocation();
        // Generate unique alert ID
        String alertID = generateAlertID();
        // Build Firebase JSON payload
        String payload = buildPayload(
            pendingSource,
            alertID,
            loc
        );
        Serial.println("Sending alert: " + alertID);
        // Upload alert metadata to Firebase
    }
}

```

```
if (sendAlertHTTP(payload, alertID)) {  
    // Alert successfully uploaded  
    beep(2, 80, 80);  
    // Trigger ESP32-CAM image capture  
    triggerCamera(alertID);  
    lastAlertID = alertID;  
    awaitingAck = true;  
  
} else {  
  
    // Offline fallback emergency mode  
    emergencyOfflineMode(alertID);  
}  
// Reset pending alert state  
alertPending = false;  
cancelPressCount = 0;  
lastTriggerTime = millis();  
digitalWrite(LED_PIN, LOW);
```